Des traces aux agents stateful

Orchestration d'agents & Monitoring : LangGraph + Langfuse · Jour 2

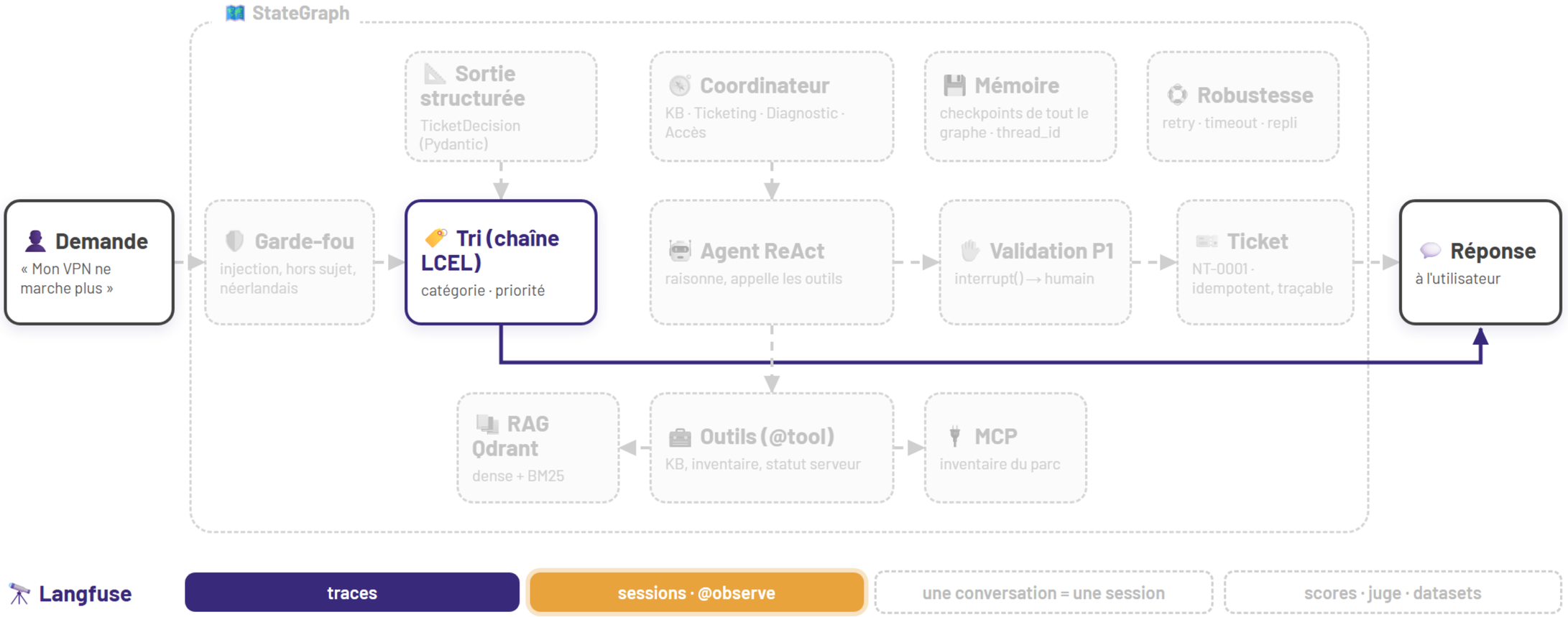
Vendredi 06/11/2026 · reprise après la slide 57 du jour 1 · Stack 100 % locale



bstorm

Où en est HelpDesk-Ops ?

Hier : une chaîne de tri, tracée dans Langfuse. Ce matin : regrouper les traces et instrumenter son propre code.



Chaque brique laisse sa trace : l'arbre d'observations grandit avec le système.

construit · ce module · à venir

Deux fichiers aujourd'hui

Ce qui est projeté, et ce que vous gardez ouvert pendant les labs

📽️ Le deck présenté

- les concepts, les schémas, les démos
- les consignes de chaque lab et exercice
- la carte d'architecture au début de chaque partie

📎 L'annexe : les fiches de référence

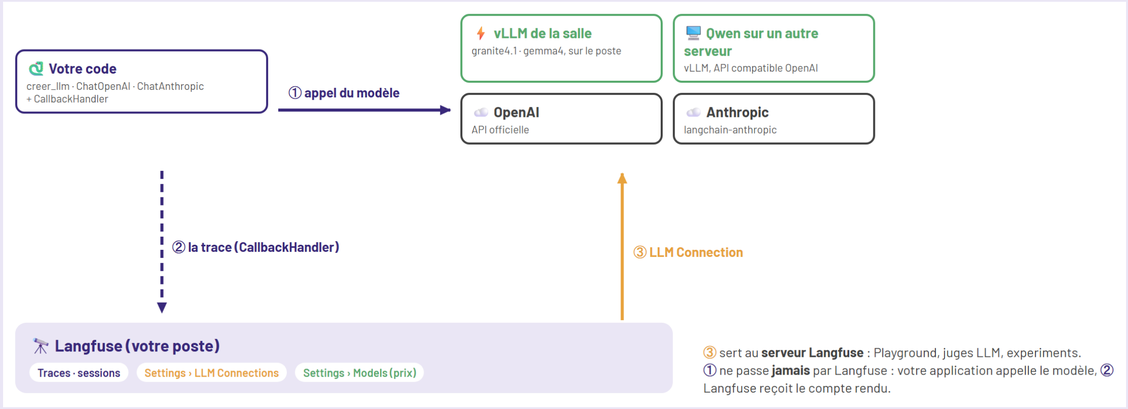
- toutes les fiches 🔎 : import, paramètres, retour, pièges
- les approfondissements (pièges classés, variantes, exercice 1.2)
- à garder ouverte pendant les labs

Toute la syntaxe d'un exercice est dans l'annexe, rangée dans l'ordre du cours. **La slide de chaque partie indique les pages de ses fiches.**

01

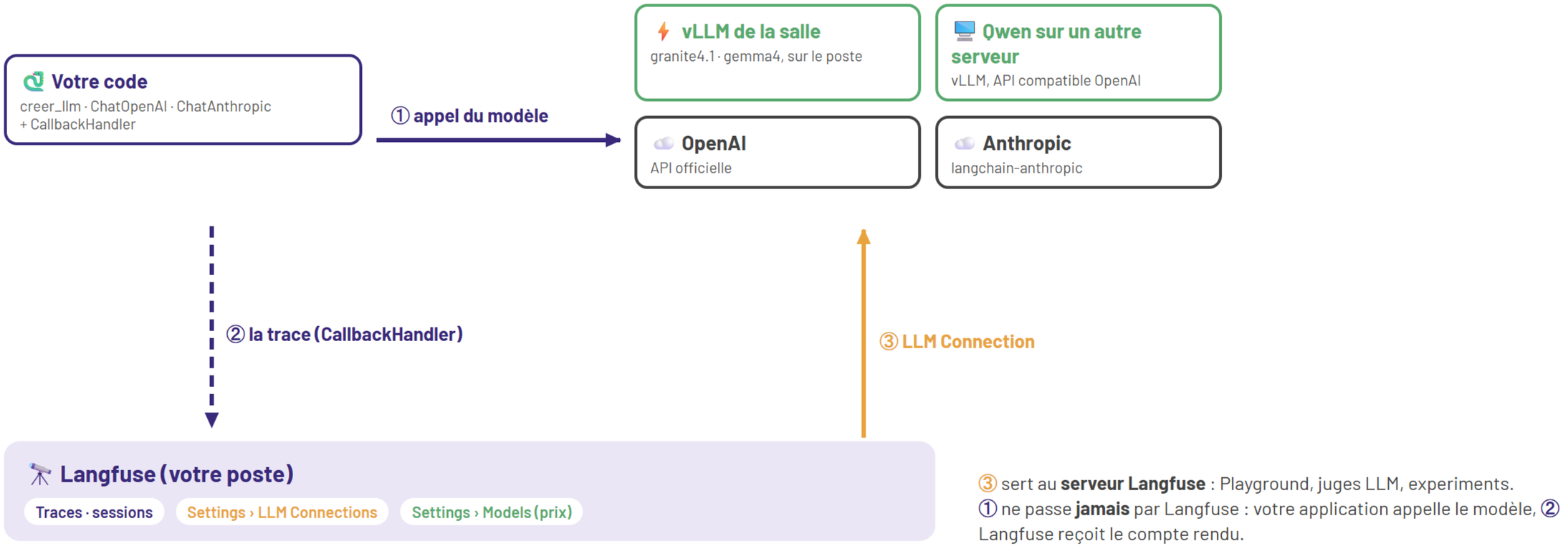
Vos questions du jour 1

Brancher un autre modèle (OpenAI, Anthropic, un Qwen sur un autre serveur), par le code et dans Langfuse ; et Langflow.



Brancher un autre modèle : deux portes d'entrée

Le code de votre application d'un côté, les réglages du serveur Langfuse de l'autre



③ sert au **serveur Langfuse** : Playground, juges LLM, experiments.
① ne passe **jamais** par Langfuse : votre application appelle le modèle, ② Langfuse reçoit le compte rendu.

Par le code (1/3) : tout serveur compatible OpenAI

OpenAI, ou un Qwen servi par vLLM sur un autre serveur : la même classe, le même handler

```python
import os
from langchain_openai import ChatOpenAI
from langfuse.langchain import CallbackHandler

# OpenAI : l'API officielle (OPENAI_API_KEY dans le .env)
gpt = ChatOpenAI(model=os.environ["OPENAI_MODEL"], temperature=0)

# Qwen servi par vLLM sur un autre serveur : même classe
qwen = ChatOpenAI(
    model="Qwen/Qwen3-8B",                  # --served-model-name
    base_url="http://10.0.0.42:8000/v1",    # le serveur, pas le poste
    api_key=os.environ["QWEN_API_KEY"],     # vllm serve --api-key
    temperature=0, max_retries=0,
)

handler = CallbackHandler()                 # LE MÊME pour tous
for llm in (gpt, qwen):
    llm.invoke("Classe : mon VPN ne marche plus",
               config={"callbacks": [handler]})
```

Ne change pas

- le CallbackHandler, propagate_attributes, @observe
- la trace part du poste, quel que soit le modèle

Change

- le nom du modèle dans la trace : celui passé à model=
- le coût : seulement si Settings › Models connaît ce nom
- le raisonnement de Qwen3 n'est pas récupéré par ChatOpenAI

Par le code (2/3) : Anthropic avec langchain-anthropic

La classe native ; et la variante « compatible OpenAI », réservée aux essais

```
# lock gelé sur langchain-core 1.6 : la 1.7.x exige core >= 1.7.5
uv add --project infrastructure "langchain-anthropic==1.6.1"
```

```
from langchain_anthropic import ChatAnthropic

claude = ChatAnthropic(
    model="claude-sonnet-5-5",          # ANTHROPIC_API_KEY lue dans l'env
    max_tokens=1024,
)
claude.invoke("Classe : mon VPN ne marche plus",
              config={"callbacks": [handler]})

# Variante sans nouveau paquet : l'endpoint compatible OpenAI
compat = ChatOpenAI(model="claude-sonnet-5-5",
                    base_url="https://api.anthropic.com/v1/",
                    api_key=os.environ["ANTHROPIC_API_KEY"])
```

Pourquoi la classe native ?

- même interface : invoke, stream, bind_tools
- toutes les fonctions du fournisseur (cache de prompt, raisonnement)

L'endpoint compatible OpenAI

- Anthropic le destine aux tests et comparaisons, pas à la production
- pas de cache de prompt ; tool_choice « auto » seulement sur les modèles 5.5

Par le code (3/3) : un Qwen sur un autre serveur, dans le cours

Le serveur sert le modèle ; le .env du poste le désigne ; creer_llm ne change pas

serveur

```
# Sur le serveur GPU : vLLM, API compatible OpenAI
vllm serve Qwen/Qwen3-8B --api-key "$QWEN_API_KEY" \
    --enable-auto-tool-choice --tool-call-parser hermes \
    --host 0.0.0.0 --port 8000
```

poste

```
# infrastructure/.env du poste : creer_llm("agent") suit
LLM_BACKEND=vllm
VLLM_URL_AGENT=http://10.0.0.42:8000/v1
MODEL_AGENT=Qwen/Qwen3-8B
VLLM_API_KEY=<la clé du serveur Qwen>
```

vérifier

```
curl -s -H "Authorization: Bearer $QWEN_API_KEY" \
     http://10.0.0.42:8000/v1/models        # le poste voit-il le serveur ?
```

Côté réseau

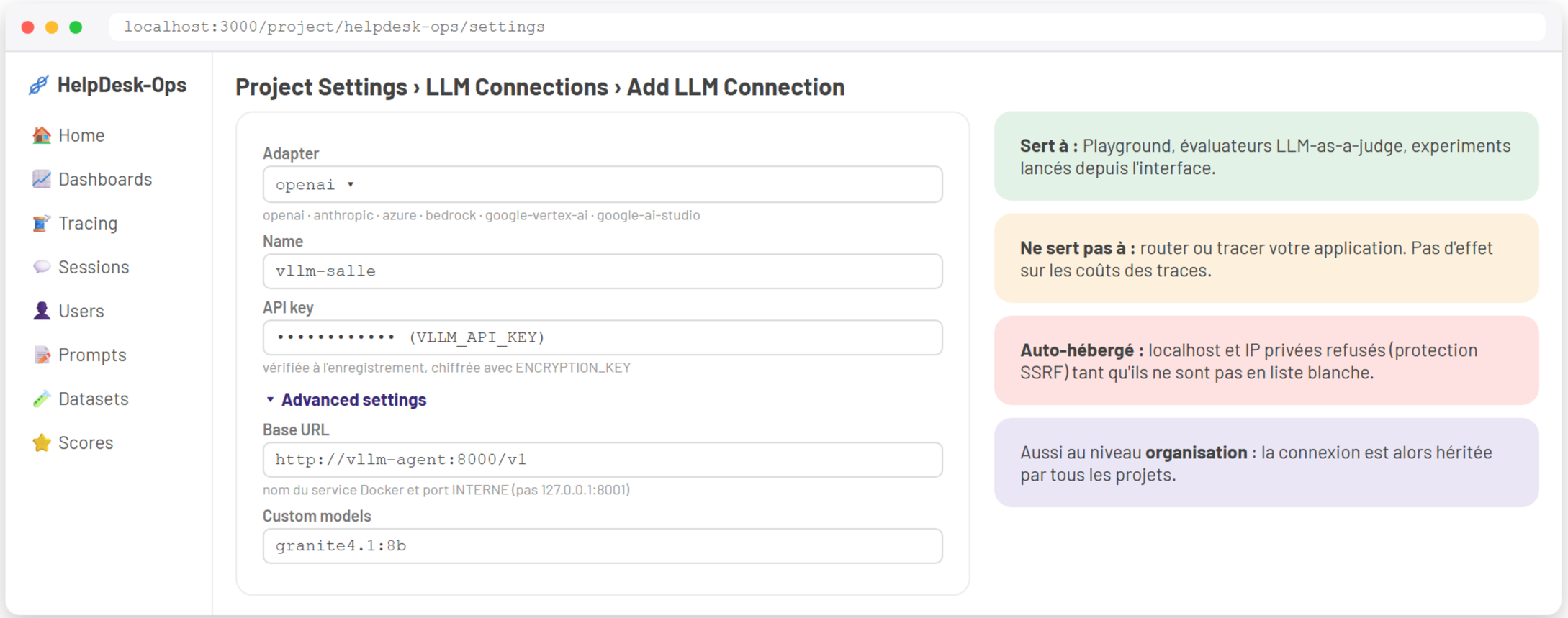
- le port 8000 ouvert du poste vers le serveur, rien d'autre
- clé API obligatoire ; HTTPS hors du réseau local

Pièges

- VLLM_API_KEY sert à tous les rôles : si les clés diffèrent, small et judge cassent
- Qwen3 réfléchit par défaut : creer_llm(reasoning=False)

Dans l'interface (1/3) : Settings › LLM Connections

Donner un modèle au serveur Langfuse : Playground, juges LLM, experiments



Maquette pédagogique de l'interface Langfuse v4 (disposition simplifiée)

Dans l'interface (2/3) : pourquoi votre vLLM est refusé

Langfuse auto-hébergé se protège des requêtes vers le réseau interne (SSRF)

```
.env

# infrastructure/.env (lu par langfuse-web ET langfuse-worker)
LANGFUSE_LLM_CONNECTION_WHITELISTED_HOST=vllm-agent,vllm-small,vllm-juge
# un Qwen sur un autre serveur du réseau local :
LANGFUSE_LLM_CONNECTION_WHITELISTED_IPS=10.0.0.42
```

```
appliquer

docker compose -f docker-compose.yml -f docker-compose.vllm.yml up -d
# recrée langfuse-web et langfuse-worker avec les nouvelles variables
```

La règle

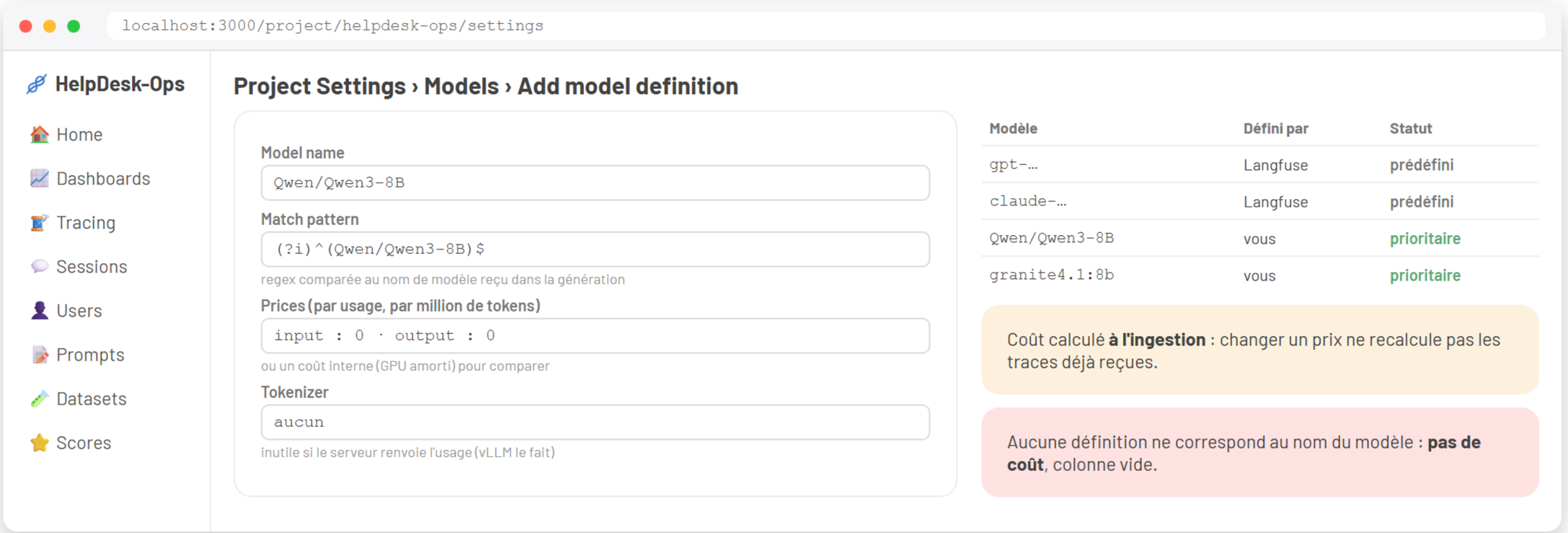
- Base URL vérifiée à l'enregistrement, au test, à chaque appel
- localhost, 127.0.0.1, IP privées : refusés
- pas d'interrupteur global : des listes blanches

Quelle URL ?

- Langfuse tourne dans Docker : 127.0.0.1, c'est le conteneur lui-même
- salle : http://vllm-agent:8000/v1

Dans l'interface (3/3) : Settings › Models, le prix d'un modèle

Langfuse calcule le coût à partir de l'usage reçu et d'une définition de modèle



Maquette pédagogique de l'interface Langfuse v4 (disposition simplifiée)

🔒 Un modèle externe : ce qui sort de la machine

Le contrat « 100 % local » du cours est un choix ; avec un fournisseur, il devient un contrat écrit

📤 Le prompt part chez le fournisseur

le ticket entier : nom, e-mail, poste. Le masque Langfuse protège la TRACE, pas l'appel au modèle.

🥠 Masquer avant l'appel

appliquer masquer_pii à l'entrée du modèle ; minimisation (RGPD art. 5)

⚖️ Les juges aussi

une LLM Connection OpenAI ou Anthropic envoie au fournisseur les traces et datasets évalués

📜 Le cadre

contrat de sous-traitance (art. 28), transferts hors UE (chap. V), conservation chez le fournisseur

🔑 La clé API

un secret : le .env, jamais le notebook ni une trace

Langflow : où le placer ?

Un éditeur visuel pour prototyper ; LangGraph en code pour livrer

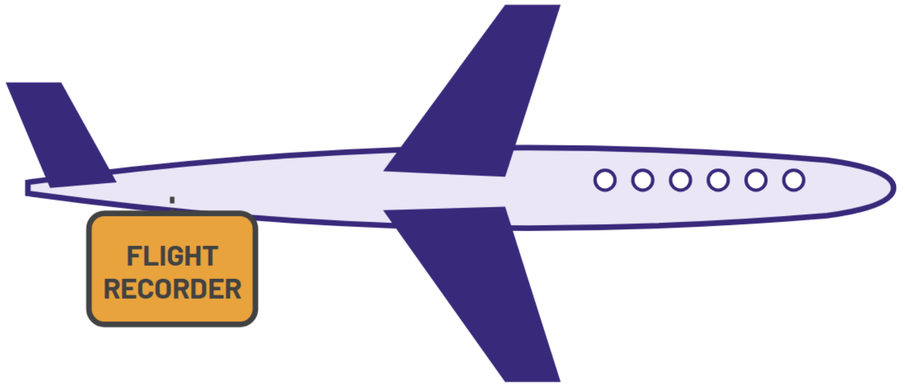
	Langflow	LangGraph en code (ce cours)
Construire	glisser-déposer des composants dans un éditeur visuel	Python : état, nœuds, arêtes
Versionner	un flow exporté en JSON	git : un diff lisible, une revue de code
Tester	le Playground, à la main	pytest, datasets, experiments, gate de CI
Contrôler	les composants fournis	reducers, interrupt(), RetryPolicy, sous-graphes
Observer	Langfuse par variables d'environnement du serveur Langflow	CallbackHandler, @observe, sessions
Exposer	API d'exécution des flows, serveur et client MCP	votre API, langgraph dev, MCP

Langflow pour dessiner une idée en une heure et la montrer au métier ; LangGraph pour la livrer, la tester et la maintenir. **Le visuel du cours : le dessin de VOTRE graphe (draw_mermaid), sa trace, et la carte de chaque partie.**

02

Observer dès la première chaîne

Langfuse v4 : tracer la chaîne qu'on vient d'écrire, les sessions, votre propre code, et la protection des données.



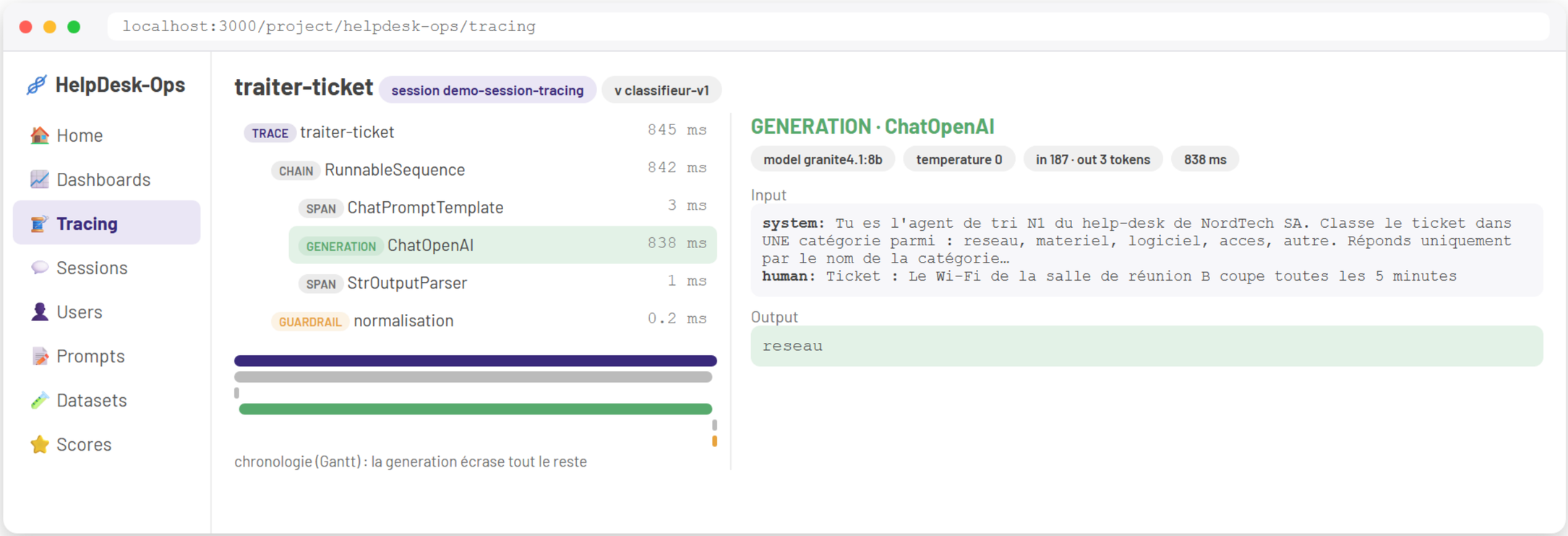
enregistre **tout**, **dès le 1er vol**... pas après l'incident

✈️ **Boîte noire**
altitude · vitesse · commandes · voix cockpit

🪢 **Langfuse**
prompts · réponses · outils · tokens · latence

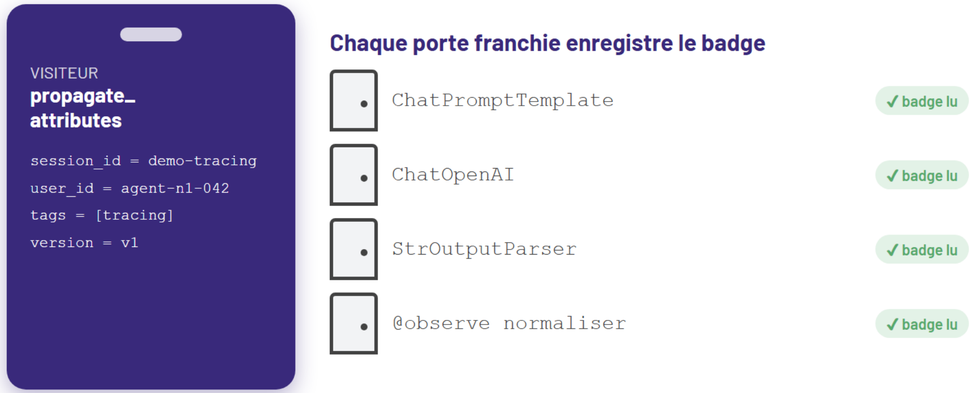
📎 Fiches : annexe p. 2–8

Lire une trace dans l'UI



Maquette pédagogique de l'interface Langfuse v4 (disposition simplifiée)

Sessions, utilisateurs, tags : propagate_attributes



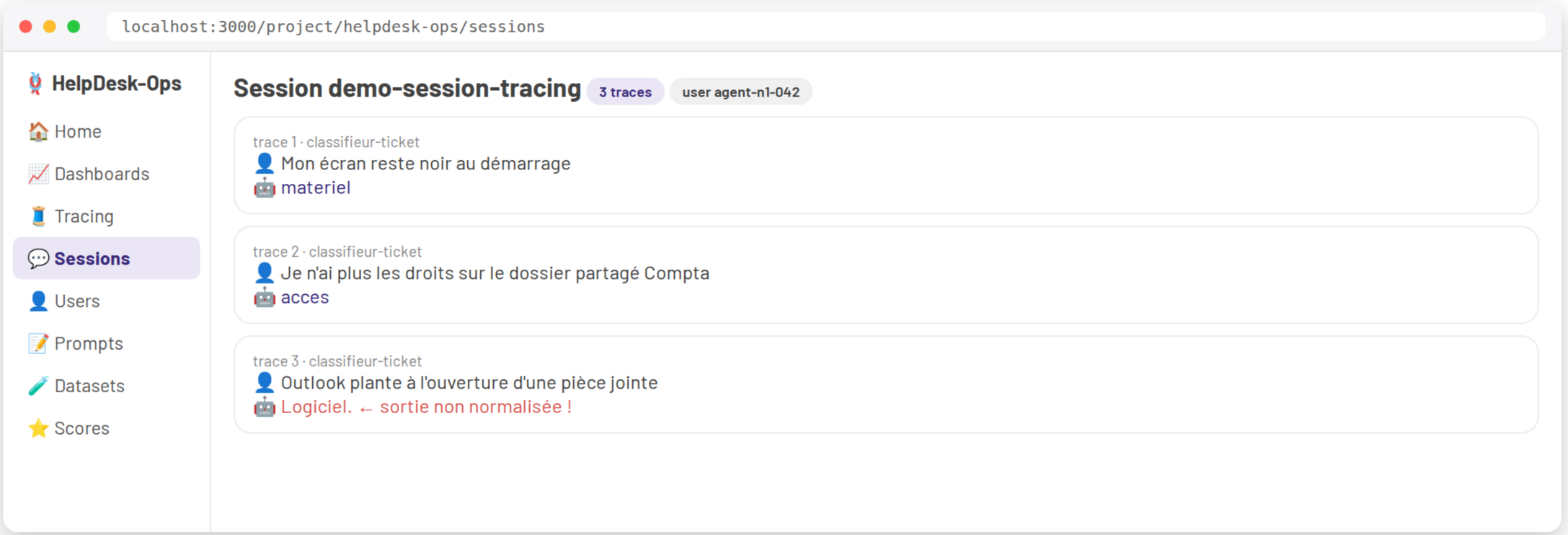

```python
from langfuse import propagate_attributes

cfg = {"callbacks": [langfuse_handler]}
with propagate_attributes(
    session_id="demo-session-tracing",     # regroupe les traces
    user_id="agent-n1-042",           # pseudonyme, jamais un nom
    tags=["tracing", "classifieur"],
    metadata={"module": "tracing"},   # dict[str, str] !
    version="classifieur-v1",         # comparaisons de versions
):
    for ticket in tickets_session:
        classifieur.invoke({...}, config=cfg)
```

💡 Le « Tips » du Formateur

v4 : metadata est un dict[str, str] dont chaque valeur fait au plus 200 caractères. Convertissez les entiers en str. Et jamais de donnée personnelle dans un tag : les tags servent à filtrer des tableaux de bord partagés.

Une conversation = une session



Maquette pédagogique de l'interface Langfuse v4 (disposition simplifiée)

Instrumenter votre propre code : @observe

```python
from langfuse import observe

@observe(name="traiter-ticket")                    # observation racine
def traiter_ticket(ticket: str) -> dict:
    brute = classifieur.invoke(                    # LangChain, imbriqué dessous
        {"ticket": ticket, "categories": ", ".join(CATEGORIES)},
        config={"callbacks": [langfuse_handler]})

    with langfuse.start_as_current_observation(    # étape métier typée
            name="normalisation", as_type="guardrail", input=brute) as etape:
        categorie = normaliser_categorie(brute)    # 'Logiciel.' -> 'logiciel'
        etape.update(output=categorie)

    file = "N2-infra" if categorie in ("reseau", "materiel") else "N1"
    return {"categorie": categorie, "file": file}
```

Pourquoi typer ?

- icônes et filtres par type
- guardrail, tool, retriever…
- lisibilité dès 4 agents (module Multi-agents)

👀 À observer

Le code Python prend quelques µs, le LLM des centaines de ms : on optimise les appels LLM d'abord.

🔒 RGPD : ce qui entre dans une trace est une donnée personnelle

Obligation RGPD	Traduction technique
Minimisation (art. 5.1.c)	masquer avant envoi (mask=), ne tracer que l'utile
Privacy by design (art. 25)	masque configuré à la création du client
Limitation de conservation (art. 5.1.e)	durée définie et purge planifiée (rétention auto : Langfuse Enterprise)
Sécurité (art. 32)	UI non exposée, accès restreint, secrets hors du code

🏛️ Et NIS2 ?

Les traces servent de preuve pour analyser et notifier un incident : à conserver assez longtemps, mais pas trop, et sans données superflues.

💡 Le « Tips » du Formateur

Le masquage par regex est une ceinture, pas un parachute : il rate les noms propres, les adresses postales et les fautes de frappe. En entreprise, combinez regex, détection d'entités nommées (Presidio), rétention courte et accès restreint. Aucune de ces mesures ne suffit seule.

Avant de quitter : flush()

```
# notebook, script batch, job de CI
langfuse.flush()

# application (FastAPI, Streamlit), à l'arrêt
langfuse.shutdown()
```

Pourquoi ?

- le SDK regroupe les observations et les envoie en arrière-plan
- objectif : ne jamais ralentir votre application
- conséquence : les derniers événements d'un script court peuvent être perdus

Cause n°1 de

« mes traces n'apparaissent pas »

Premier réflexe avant d'ouvrir un ticket : ajouter flush().

🧪 Mini-défi : tracer une étape métier

En autonomie : si vous avez de l'avance, ou après la journée (corrigé distribué)

- **1.** Écrire estimer_priorite(ticket), qui renvoie « P1 » si le ticket contient « bloqué », « urgent », « production » ou « tous les utilisateurs », sinon « P3 »
- **2.** La tracer comme une observation de type tool
- **3.** Créer traiter_ticket_v2 qui renvoie aussi la priorité
- **4.** Traiter 3 tickets dans la session defi-j1-<initiales>, version classifieur-v2
- **5.** Dans l'UI : identifier l'étape la plus lente de chaque trace

📂 Fichiers

- exercices/enonces/J1_01_defi.ipynb
- corrigé : exercices/corriges/

Pour aller plus loin

« Je ne suis pas bloqué » déclenche aussi P1 : comment corriger la règle ? Que coûterait le recours au LLM ?

🛠️ La construction dirigée

Le même système de tickets, brique par brique, en scripts Python : vous le voyez grandir

```
# 0. dans chaque terminal, depuis la racine du cours
PY="uv run --project infrastructure python"

# 1. une seule fois : l'étape 0, distribuée, devient votre fichier
cp etape_00_chaine_tri.py construction/atelier/mon_helpdesk.py

# 2. à chaque étape : vous recopiez les blocs NOUVEAU
#    que le formateur écrit à l'écran, puis vous lancez
$PY construction/atelier/mon_helpdesk.py

# 3. en fin d'étape : la correction est distribuée
```

📄 Un fichier par étape

- complet et exécutable : tout le système à la fin de l'étape

🧱 Seul le NOUVEAU change

- # ═══ NOUVEAU (étape N) : le reste est identique

🛟 Perdu ? On repart

- la correction de l'étape, distribuée à la fin, remplace votre fichier

🛠️ Construction · étape 1 : sessions et @observe

correction distribuée en fin d'étape : etape_01_sessions_observe.py · environ 10 min

```
lancer

$PY construction/atelier/mon_helpdesk.py
```

```
perdu ? repartir de la correction

# en fin d'étape, avec la correction distribuée :
cp etape_01_sessions_observe.py \
   construction/atelier/mon_helpdesk.py
$PY construction/atelier/mon_helpdesk.py
```

🧱 Ce qu'on ajoute

- @observe : traiter_ticket devient la racine de la trace
- propagate_attributes : session, utilisateur, tags

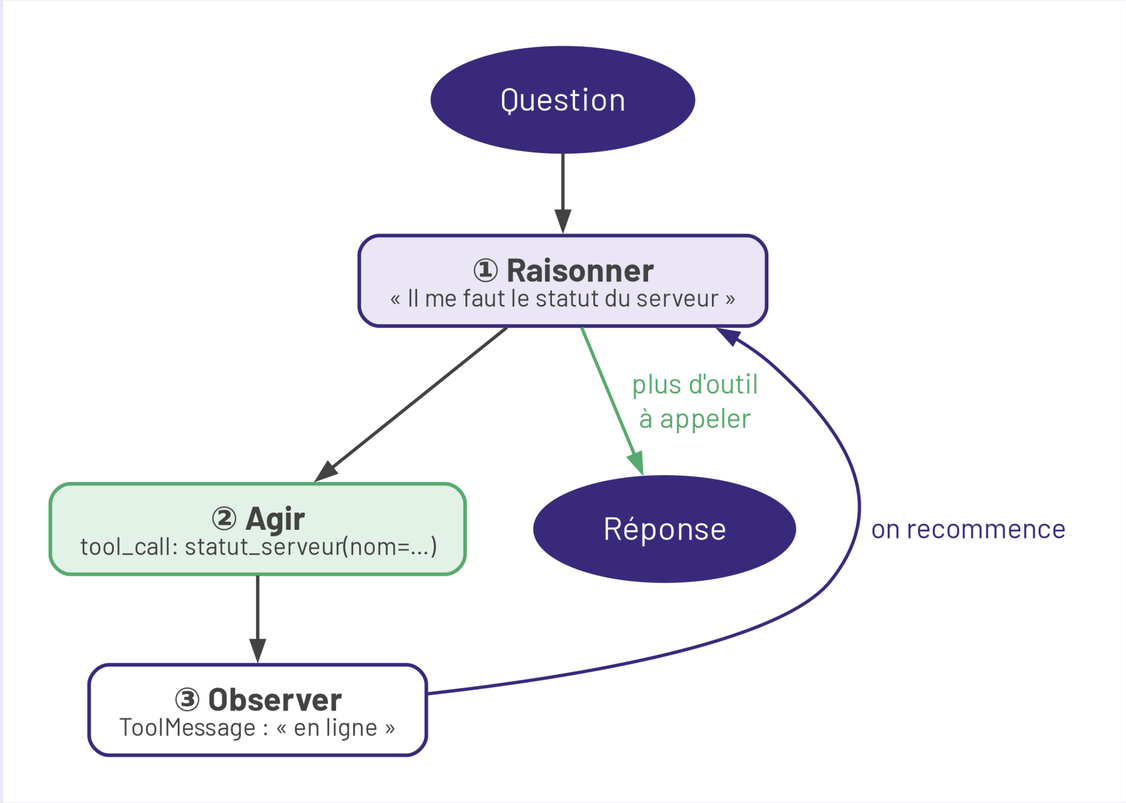
👀 À observer

- Tracing : 2 traces traiter-ticket, la chaîne DEDANS
- Sessions : construction-etape-1
- Users : agent-n1-042

03

Les fondamentaux des agents

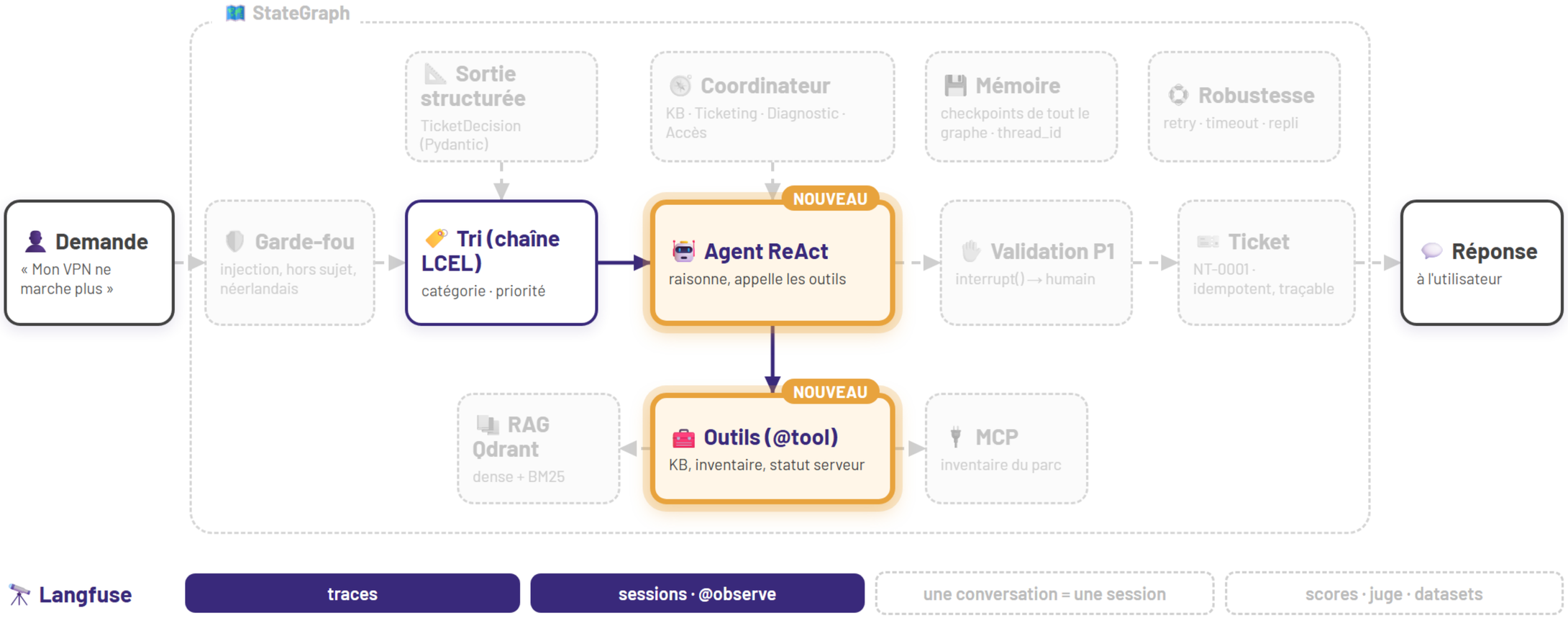
Les outils, la boucle ReAct codée à la main, puis create_agent, en observant ce que fait réellement l'agent.



📎 Fiches : annexe p. 9–16

Où en est HelpDesk-Ops ?

On ajoute l'agent et ses outils : il décide lui-même d'appeler la KB, l'inventaire ou la supervision.



Chaque brique laisse sa trace : l'arbre d'observations grandit avec le système.

construit · ce module · à venir

Le socle helpdesk_ops : la première cellule de chaque notebook

La version de référence du code de la matinée est rangée dans src/helpdesk_ops : tout le monde importe la même

```
import os
from langfuse import observe, propagate_attributes

from helpdesk_ops.config import creer_llm
from helpdesk_ops.observabilite import init_observabilite
from helpdesk_ops.texte import sans_accents

langfuse, langfuse_handler = init_observabilite()   # .env + masque PII
llm = creer_llm("agent")                            # temperature=0
```

Module	Ce qu'il apporte
config	creer_llm(rôle), charger_env()
observabilite	init_observabilite() → (client, handler)
pii	masquer_pii, le masque de la partie 4
texte	sans_accents, normaliser
tri	le classifieur, normaliser_categorie
donnees	INVENTAIRE, KB… (fictifs)

💡 Le « Tips » du Formateur

Paquet installé en mode éditable par uv sync : une modification dans src/helpdesk_ops est prise en compte après Kernel → Restart. Le client Langfuse est un singleton : en cas de doute, redémarrez aussi le kernel.

Anatomie d'un outil (@tool)

```python
from langchain_core.tools import tool

@tool(parse_docstring=True)
def statut_serveur(nom: str) -> str:
    """Retourne le statut (en ligne / hors ligne) d'un serveur
    de l'inventaire NordTech.

    Args:
        nom: nom d'hôte exact, ex. 'srv-mail-01'
    """
    inventaire = {"srv-mail-01": "en ligne", "srv-erp-02": "hors ligne"}
    return inventaire.get(nom, "inconnu")
```

Nom de la fonction
→ nom de l'outil vu par le LLM

Type hints
→ schéma JSON des arguments

Docstring
→ description : c'est le contrat, le LLM ne lit que cela

Args:
→ description de chaque paramètre

Retour str
→ ToolMessage renvoyé au LLM

Un outil se teste sans LLM, et répond par une erreur explicite

```python
TICKETS_OUVERTS = {"N1": 14, "N2-infra": 6, "N2-applicatif": 3}

@tool(parse_docstring=True)
def tickets_ouverts(file: str) -> str:
    """Retourne le nombre de tickets ouverts dans une file du help-desk.

    Args:
        file: nom exact de la file : 'N1', 'N2-infra' ou 'N2-applicatif'
    """
    if file not in TICKETS_OUVERTS:
        return (f"file inconnue : '{file}'. "
                f"Files valides : {', '.join(TICKETS_OUVERTS)}")
    return f"{TICKETS_OUVERTS[file]} tickets ouverts dans la file {file}"

print(tickets_ouverts.invoke({"file": "N2-infra"}))       # un dict d'arguments
print(tickets_ouverts.invoke({"file": "infra niveau 2"}))
```

▶ **Sortie**

```
6 tickets ouverts dans la file N2-infra
file inconnue : 'infra niveau 2'. Files valides : N1, N2-infra, N2-applicatif
```

Un outil est un Runnable

- .invoke({argument: valeur})
- .name, .description, .args : ce que verra le LLM

Pourquoi une erreur explicite ?

Le texte repart au LLM, qui peut se corriger au tour suivant. Un None ou une exception le laisse sans piste.

Ce que le LLM voit réellement

bind_tools convertit l'outil en schéma JSON envoyé avec chaque requête

```
{
  "type": "function",
  "function": {
    "name": "statut_serveur",
    "description": "Retourne le statut (en ligne / hors ligne)
                    d'un serveur de l'inventaire NordTech.",
    "parameters": {
      "type": "object",
      "properties": {
        "nom": {"type": "string",
                "description": "nom d'hôte exact, ex. 'srv-mail-01'"}
      },
      "required": ["nom"]
    }
  }
}
```

💡 Le « Tips » du Formateur

Sans parse_docstring=True, la section Args: n'est pas transmise : l'argument nom arrive sans description. Vérifiez avec convert_to_openai_tool(outil).

🛡️ OWASP ASI02

Chaque outil exposé est une surface d'attaque : exposez le minimum.

bind_tools : le LLM demande, votre code exécute

Le chef dicte, le commis exécute : le modèle ne touche jamais aux outils

```
TOOLS = {t.name: t for t in (statut_serveur, tickets_ouverts)}
llm_outils = llm.bind_tools(list(TOOLS.values()))   # schémas joints à chaque requête

reponse = llm_outils.invoke("Le serveur srv-erp-02 répond-il ?")
print("content    :", repr(reponse.content))
print("tool_calls :", reponse.tool_calls)

# Exécuter la demande : c'est VOTRE code qui appelle l'outil
appel = reponse.tool_calls[0]
TOOLS[appel["name"]].invoke(appel["args"])          # 'hors ligne'
```

▶ **Sortie**

```
content    : ''
tool_calls : [{'name': 'statut_serveur', 'args': {'nom': 'srv-erp-02'},
               'id': '…', 'type': 'tool_call'}]
```

Lire un tool_call

- "name" : l'outil demandé
- "args" : le dict d'arguments
- "id" : à recopier dans le ToolMessage

Pas de tool_calls ?

Modèle sans capacité « tools », ou vLLM sans —tool-call-parser : la demande reste dans content.

La conversation d'un agent, message par message

1. SystemMessage
`Tu es l'agent support N1 de NordTech SA.`

2. HumanMessage
`srv-mail-01 est-il en ligne ?`

3. AIMessage
`content="" · tool_calls=[{name: 'statut_serveur', args: {nom: 'srv-mail-01'}, id: 'call_1'}]`

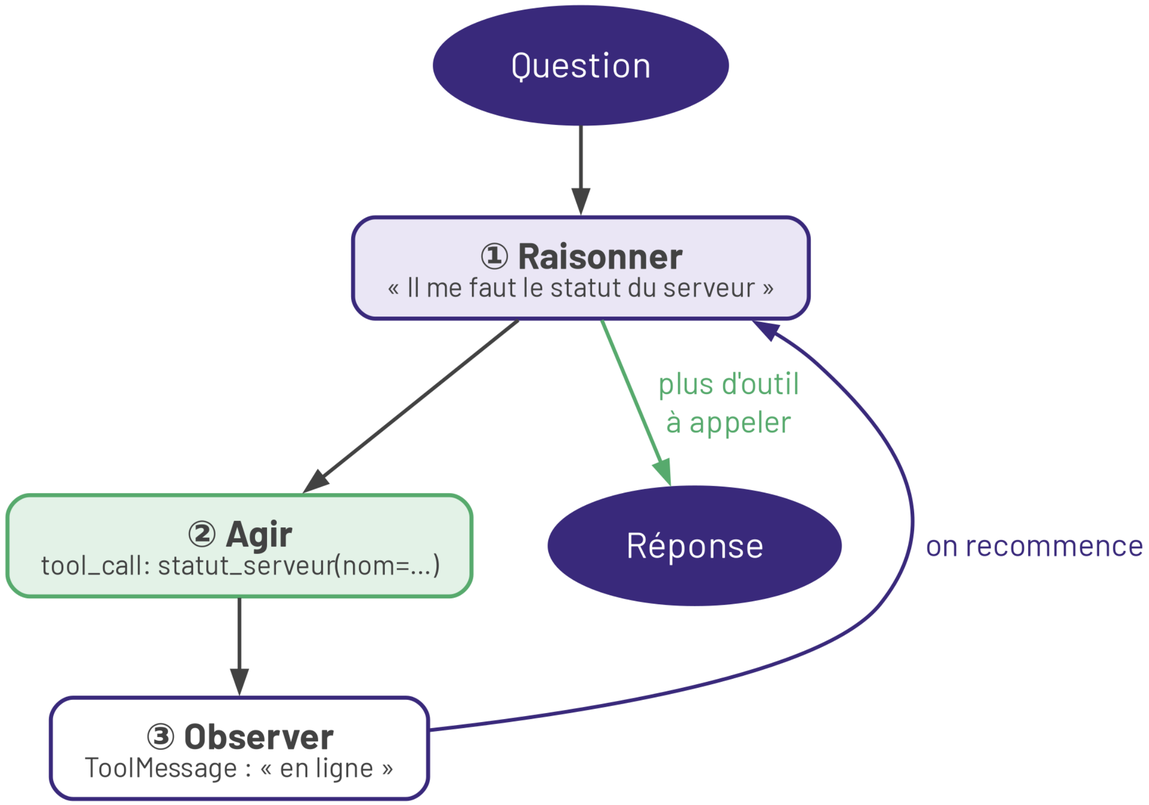
4. ToolMessage
`content='en ligne' · tool_call_id='call_1'`

5. AIMessage
`« Le serveur srv-mail-01 est en ligne. »`

⚠️ tool_call_id relie la réponse de l'outil à la demande

L'historique entier est renvoyé au LLM à chaque tour

La boucle ReAct : Raisonner → Agir → Observer



- **ReAct** (Reason + Act, Yao et al., 2022) : le modèle alterne raisonnement et actions
- **Condition d'arrêt :** l'AIMessage ne contient plus de tool_calls
- **Risque :** une boucle sans fin si le modèle redemande toujours un outil
- **Parade :** une limite de tours explicite (et recursion_limit en LangGraph)

La boucle ReAct, codée à la main

```python
from langchain_core.messages import HumanMessage, SystemMessage, ToolMessage

TOOLS = {"statut_serveur": statut_serveur}
llm_outils = llm.bind_tools([statut_serveur])

messages = [SystemMessage("Tu es l'agent support NordTech."),
            HumanMessage("srv-mail-01 est-il en ligne ?")]

for tour in range(5):                          # garde-fou : jamais de boucle infinie
    reponse = llm_outils.invoke(messages, config={"callbacks": [langfuse_handler]})
    messages.append(reponse)
    if not reponse.tool_calls:                 # plus d'outil demandé -> réponse finale
        break
    for appel in reponse.tool_calls:           # exécuter chaque outil demandé
        resultat = TOOLS[appel["name"]].invoke(appel["args"])
        messages.append(ToolMessage(content=resultat, tool_call_id=appel["id"]))

print(reponse.content)                         # « srv-mail-01 est en ligne. »
```

15 lignes

- boucle
- arrêt
- exécution
- historique

Ce qui manque

- reprise après crash
- pause humaine
- parallélisme

La même chose en un seul appel : create_agent (LangChain 1.x)

```python
from langchain.agents import create_agent

agent = create_agent(
    model=llm,                        # le modèle de LangChain express
    tools=[statut_serveur],
    system_prompt="Tu es l'agent support N1 de NordTech SA.",
)

resultat = agent.invoke(
    {"messages": [{"role": "user",
                   "content": "srv-mail-01 est-il en ligne ?"}]},
    config={"callbacks": [langfuse_handler],
            "recursion_limit": 10},
)
print(resultat["messages"][-1].content)
```

Sous le capot

- un StateGraph LangGraph compilé
- nœuds model et tools
- la boucle de la slide précédente

Et en plus (jour 2)

- checkpointer
- response_format (Pydantic)
- interrupt (HITL)

Lire le résultat d'un agent : la liste des messages

Un agent prend un état {"messages": [...]} et rend l'état complété

```python
resultat = agent.invoke(
    {"messages": [{"role": "user", "content": "srv-mail-01 est-il en ligne ?"}]},
    config={"callbacks": [langfuse_handler], "recursion_limit": 10})

for message in resultat["messages"]:
    print(f"{type(message).__name__:<13} {message.content[:30]!r}",
          getattr(message, "tool_calls", "") or "")

reponse_finale = resultat["messages"][-1].content   # le dernier message
```

▶ Sortie (contenu indicatif, séquence garantie)

```
HumanMessage  'srv-mail-01 est-il en ligne ?'
AIMessage     '' [{'name': 'statut_serveur', 'args': {'nom': 'srv-mail-01'}, …}]
ToolMessage   'en ligne'
AIMessage     'srv-mail-01 est en ligne.'
```

La séquence

- Human → AI (tool_calls)
- → Tool → ... → AI (réponse)
- la boucle ReAct, à plat

type(agent)

CompiledStateGraph : un agent EST un graphe. On le dessine nous-mêmes dans la partie suivante.

Ce que Langfuse montre de l'agent



Maquette pédagogique de l'interface Langfuse v4 (disposition simplifiée)

Les 6 pièges classiques des agents locaux

🐣 Petit modèle

tool_calls absents ou mal formés. Mesurez l'effet (module Évaluation) avant de conclure.

♾️ Boucle infinie

le modèle redemande sans cesse un outil. Fixez un nombre maximal de tours ou recursion_limit.

📝 Docstring vague

« fait des trucs réseau » → mauvais outil choisi. Soyez précis et donnez des exemples.

🧰 Trop d'outils

au-delà d'une dizaine, le choix se dégrade. Regroupez-les ou déléguez à des sous-agents (module Multi-agents).

☣️ Outil dangereux

eval(), shell, SQL libre = exécution de code (OWASP ASI05). Liste blanche + validation (red-team, J4).

📦 Contexte saturé

l'historique grossit à chaque tour ; au-delà du contexte, vLLM refuse (400), Ollama tronque en silence.

🧪 À vous : J1-02

13h00–14h00 · notebook J1_02_agents_fondamentaux

- **§3** écrire statut_serveur et tickets_ouverts, les tester sans LLM, comparer le schéma avec et sans parse_docstring
- **§4** bind_tools : lire content et tool_calls
- **§5** la boucle ReAct à la main, tracée (session j1-02-react)
- **§6** create_agent : lire la liste des messages
- **Mini-défi :** un outil redemarrer_serveur qui ne redémarre rien ; l'agent l'appelle-t-il sans demander confirmation ?

Slides à revoir

- outil · tester un outil
- bind_tools · boucle ReAct
- create_agent · résultat

§7 en démo

Petit modèle contre grand modèle : regarder les arguments envoyés dans Langfuse, pas l'impression générale.

🛠️ Construction · étape 2 : l'agent et ses outils

correction distribuée en fin d'étape : etape_02_agent_outils.py · environ 10 min

lancer

```
$PY construction/atelier/mon_helpdesk.py \
    "srv-erp-02 est-il en ligne ?"
```

perdu ? repartir de la correction

```
# en fin d'étape, avec la correction distribuée :
cp etape_02_agent_outils.py \
   construction/atelier/mon_helpdesk.py
$PY construction/atelier/mon_helpdesk.py
```

🧱 Ce qu'on ajoute

- deux @tool : statut_serveur, chercher_kb
- create_agent : l'agent ReAct
- le tri passe la main à l'agent

👀 À observer

- le terminal : les outils appelés et leurs arguments
- Langfuse : génération → TOOL → génération

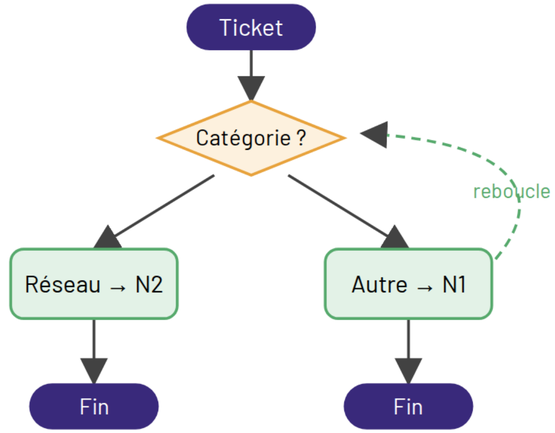
04

StateGraph pas à pas

État partagé, nœuds, arêtes, arêtes conditionnelles : le code devient un schéma.

Fiches : annexe p. 17–31

🗺️ Un organigramme de traitement



▭ Case = nœud
une fonction Python

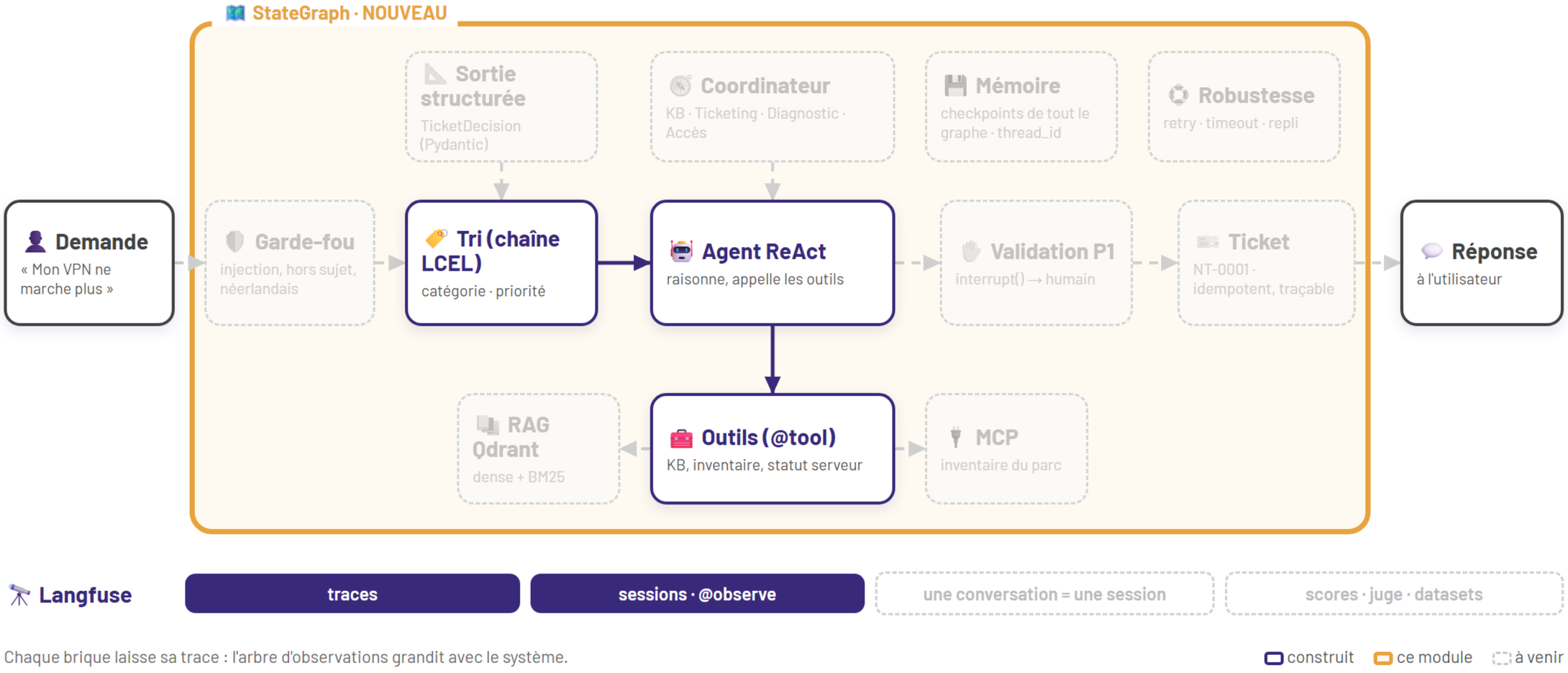
→ Flèche = arête
l'étape suivante

◇ Losange = arête conditionnelle
une fonction de routage

↺ Retour = boucle
impossible en LCEL pur

Où en est HelpDesk-Ops ?

On rend le système explicite : un StateGraph, qu'on dessine et qu'on trace.

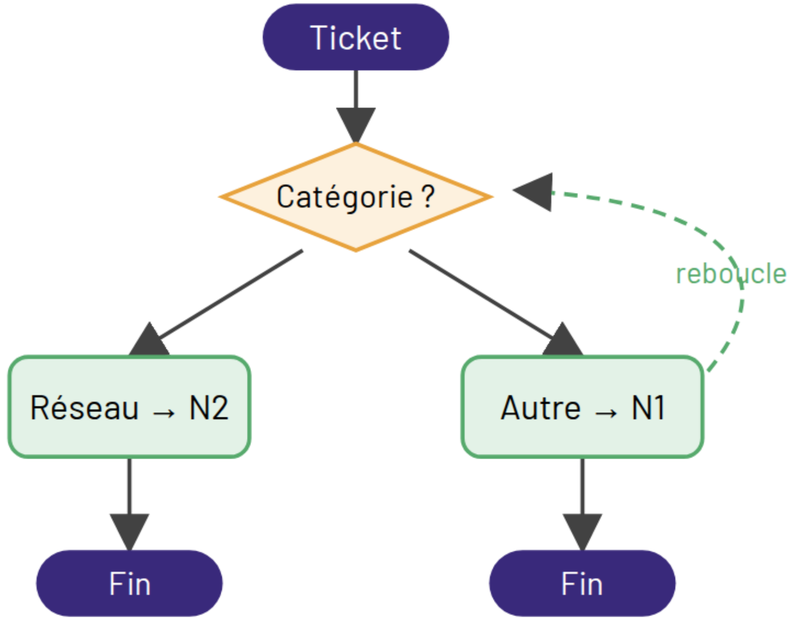


Chaque brique laisse sa trace : l'arbre d'observations grandit avec le système.

construit · ce module · à venir

Un graphe, c'est un organigramme exécutable

🗺️ Un organigramme de traitement



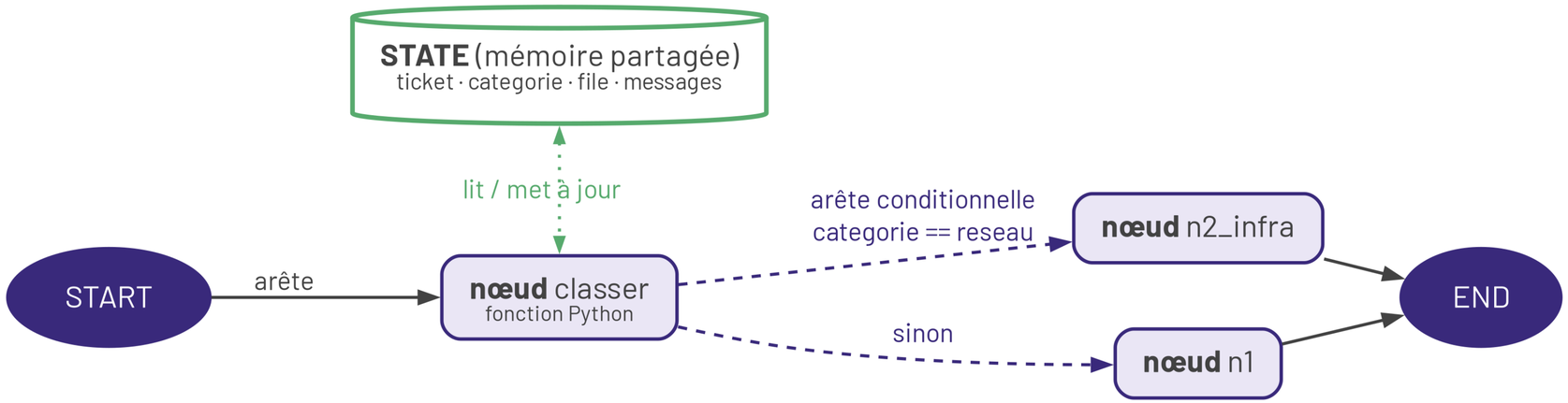
▭ Case = nœud
une fonction Python

→ Flèche = arête
l'étape suivante

◇ Losange = arête conditionnelle
une fonction de routage

↺ Retour = boucle
impossible en LCEL pur

Les 5 concepts de LangGraph



State

TypedDict partagé ; chaque nœud renvoie une mise à jour partielle

Nœud

fonction (state) → dict

Arête

add_edge(a, b) : a puis b

Arête conditionnelle

fonction de routage → nom du nœud suivant

START / END

points d'entrée et de sortie ; compile() rend le graphe exécutable

L'état et ses reducers : écraser, concaténer ou ajouter

Le reducer décide comment la mise à jour d'un nœud est fusionnée dans l'état

Sans reducer : on ÉCRASE

```
categorie: str
```

`'reseau'` + `'acces'` = `'acces'`

operator.add : on CONCATÈNE

```
etapes: Annotated[list[str], operator.add]
```

`['classer']` + `['router']` = `['classer','router']`

add_messages : on AJOUTE (ou remplace par id)

```
messages: Annotated[list, add_messages]
```

`[Human]` + `[AI]` = `[Human, AI]`

MessagesState est un TypedDict qui contient déjà messages: Annotated[list, add_messages]. C'est la base de tout agent conversationnel.

Les reducers en code : Annotated[type, reducer]

Même graphe START → a → b → END, deux déclarations de l'état

```python
import operator
from typing import Annotated

class SansReducer(TypedDict):
    journal: list[str]                              # écrase

class AvecReducer(TypedDict):
    journal: Annotated[list[str], operator.add]     # concatène

def etape_a(state) -> dict: return {"journal": ["A : ticket reçu"]}
def etape_b(state) -> dict: return {"journal": ["B : ticket classé"]}

print(construire(SansReducer).invoke({"journal": ["début"]}))
print(construire(AvecReducer).invoke({"journal": ["début"]}))
```

▶ **Sortie**

```
{'journal': ['B : ticket classé']}
{'journal': ['début', 'A : ticket reçu', 'B : ticket classé']}
```

add_messages

- ajoute les nouveaux messages
- remplace celui qui a le même id
- déjà inclus dans MessagesState

En parallèle (J3)

Deux nœuds écrivent la même clé sans reducer : InvalidUpdateError.

Premier graphe (1/2) : l'état et les nœuds

```python
from typing import Literal, TypedDict
from langgraph.graph import StateGraph, START, END

class TicketState(TypedDict):
    ticket: str
    categorie: str
    file: str

def classer(state: TicketState) -> dict:
    texte = state["ticket"].lower()
    # mise à jour PARTIELLE : seulement la clé modifiée
    return {"categorie": "reseau" if "vpn" in texte else "logiciel"}

# le type Literal sert au dessin du graphe
def router(state: TicketState) -> Literal["n2_infra", "n1"]:
    return "n2_infra" if state["categorie"] == "reseau" else "n1"

def n2_infra(state: TicketState) -> dict:
    return {"file": "N2-infra"}

def n1(state: TicketState) -> dict:
    return {"file": "N1"}
```

💡 Le « Tips » du Formateur

Un nœud renvoie uniquement les clés qu'il modifie, jamais l'état complet ni un état muté sur place. LangGraph fusionne les mises à jour via les reducers.

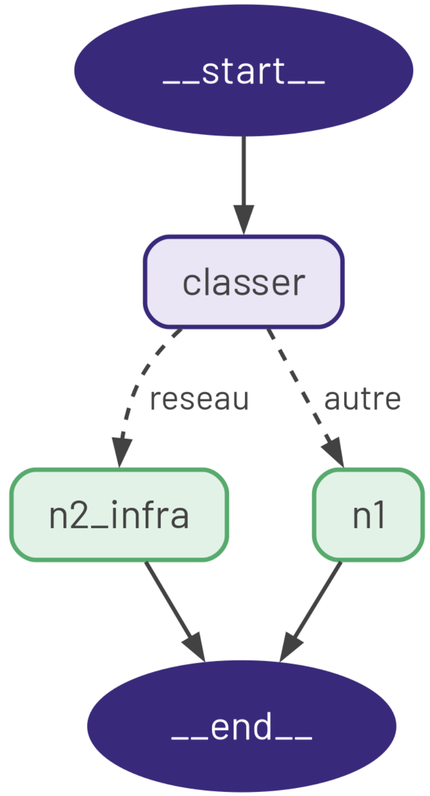
Premier graphe (2/2) : câbler, compiler, exécuter

```
builder = StateGraph(TicketState)
builder.add_node("classer", classer)
builder.add_node("n2_infra", n2_infra)
builder.add_node("n1", n1)

builder.add_edge(START, "classer")
builder.add_conditional_edges("classer", router)   # la décision
builder.add_edge("n2_infra", END)
builder.add_edge("n1", END)

graphe = builder.compile()

graphe.invoke({"ticket": "VPN coupé"})
# {'ticket': 'VPN coupé', 'categorie': 'reseau', 'file': 'N2-infra'}
```



Visualiser le graphe dans le notebook

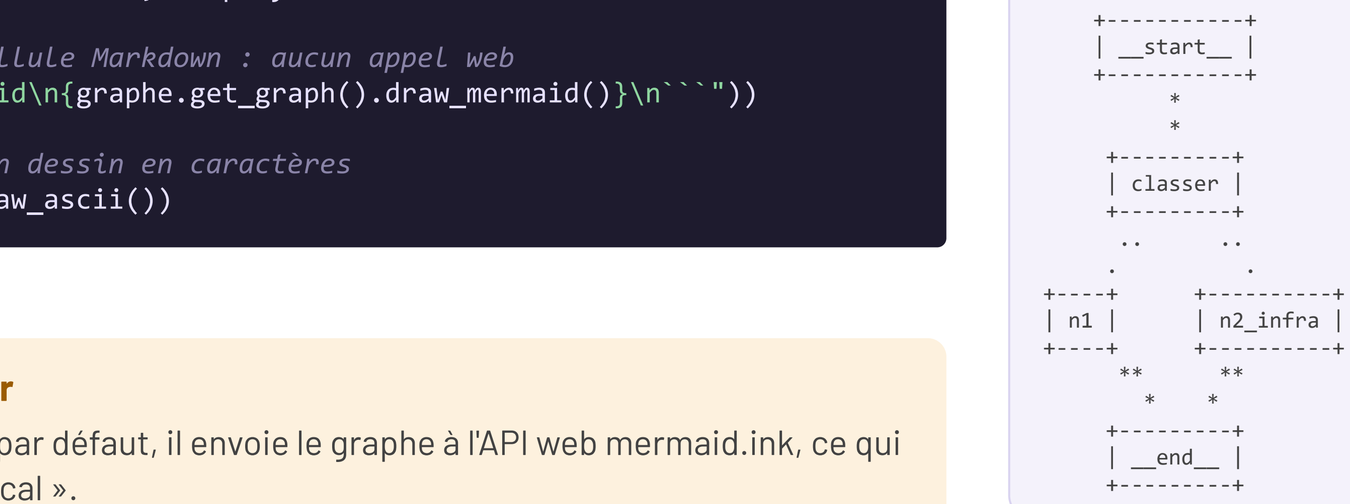
Sans quitter la machine : du texte Mermaid, que JupyterLab 4 dessine lui-même

```python
from IPython.display import Markdown, display

# Texte Mermaid dans une cellule Markdown : aucun appel web
display(Markdown(f"```mermaid\n{graphe.get_graph().draw_mermaid()}\n```"))

# Repli hors JupyterLab : un dessin en caractères
print(graphe.get_graph().draw_ascii())
```

▶ **Sortie de draw_ascii()**



💡 Le « Tips » du Formateur

Évitez draw_mermaid_png() : par défaut, il envoie le graphe à l'API web mermaid.ink, ce qui contredit le contrat « 100 % local ».

👀 **À vérifier :** les arêtes conditionnelles sont en pointillés (« .. » en ASCII). Absentes ? La fonction de routage n'est pas annotée Literal[...].

Exécuter : invoke, invoke v2 et stream

```python
# invoke classique : renvoie l'état final (dict)
etat = graphe.invoke({"ticket": "VPN coupé"})

# LangGraph ≥ 1.1 : sortie typée, interruptions accessibles (HITL)
sortie = graphe.invoke({"ticket": "VPN coupé"}, version="v2")
sortie.value        # l'état final
sortie.interrupts   # les pauses HITL en attente

# stream : suivre le graphe étape par étape
for maj in graphe.stream({"ticket": "Excel plante"}, stream_mode="updates"):
    print(maj)
# {'classer': {'categorie': 'logiciel'}}
# {'n1': {'file': 'N1'}}
```

Nouveautés 2026

- invoke / stream « v2 » typés (1.1)
- streaming v3 en bêta (1.2)

💡 Le « Tips » du Formateur

version="v2" expose .value et .interrupts : on s'en servira pour la validation humaine (module Agents stateful).

Un nœud peut appeler le LLM : le tri de tickets tracé

On remplace la règle « vpn in texte » par le classifieur LCEL : le graphe, lui, ne change pas

```python
def classer_llm(state: TicketState) -> dict:
    brute = classifieur.invoke({"ticket": state["ticket"],
                                "categories": ", ".join(CATEGORIES)})
    return {"categorie": normaliser_categorie(brute)}

def router_llm(state: TicketState) -> Literal["n2_infra", "n1"]:
    return "n2_infra" if state["categorie"] in ("reseau", "materiel") else "n1"

# câblage identique au premier graphe, avec classer_llm et router_llm -> tri

with propagate_attributes(session_id="j1-03-tri", tags=["jour-1", "stategraph"]):
    etat = tri.invoke({"ticket": "Le Wi-Fi de la salle B coupe sans arrêt"},
                      config={"callbacks": [langfuse_handler],   # une seule fois
                              "run_name": "tri-ticket"})
print(etat)
```

▶ **Sortie indicative**

```
{'ticket': 'Le Wi-Fi de la salle B coupe sans arrêt', 'categorie': 'reseau', 'file': 'N2-infra'}
```

Le handler, une fois

Passé au invoke du graphe, il suit chaque nœud et chaque appel LLM imbriqué.

👀 Dans Langfuse

La trace reproduit le graphe : classer (avec sa generation), puis n2_infra ou n1.

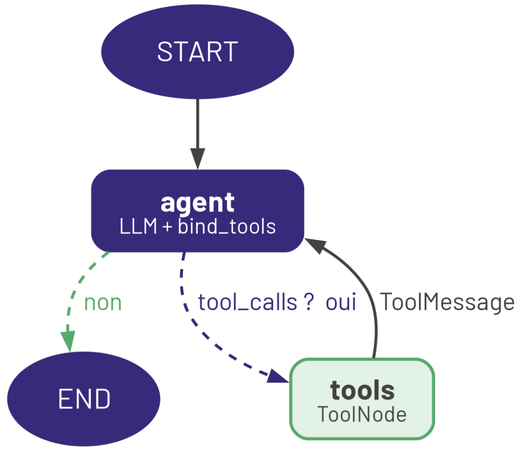
L'agent en graphe explicite (1/2) : ToolNode + tools_condition

```python
from langchain_core.messages import HumanMessage, SystemMessage
from langgraph.graph import MessagesState
from langgraph.prebuilt import ToolNode, tools_condition

OUTILS = [statut_serveur]
llm_outils = llm.bind_tools(OUTILS)
SYSTEME = SystemMessage("Tu es l'agent support N1 de NordTech SA. "
                        "Utilise les outils ; n'invente rien.")

def agent(state: MessagesState) -> dict:
    # prompt système ajouté à l'appel, pas stocké dans l'état
    return {"messages": [llm_outils.invoke([SYSTEME, *state["messages"]])]}

g = StateGraph(MessagesState)
g.add_node("agent", agent)
g.add_node("tools", ToolNode(OUTILS))               # exécute les tool_calls
g.add_edge(START, "agent")
g.add_conditional_edges("agent", tools_condition)   # -> "tools" ou END
g.add_edge("tools", "agent")                        # la boucle ReAct, dessinée
support = g.compile()
```



L'agent en graphe explicite (2/2) : l'appeler et lire sa réponse

```python
resultat = support.invoke(
    {"messages": [HumanMessage("srv-erp-02 et srv-mail-01 sont-ils en ligne ?")]},
    config={"callbacks": [langfuse_handler],
            "run_name": "agent-graphe",
            "recursion_limit": 10})

for m in resultat["messages"]:
    print(f"{type(m).__name__:<13} {m.content[:50]!r}")
```

▶ **Sortie indicative (un ou deux tours selon le modèle)**

```
HumanMessage  'srv-erp-02 et srv-mail-01 sont-ils en ligne ?'
AIMessage     ''                          <- 2 tool_calls dans le même tour
ToolMessage   'hors ligne'
ToolMessage   'en ligne'
AIMessage     'srv-erp-02 est hors ligne ; srv-mail-01 est en ligne.'
```

Entrée et sortie

- entrée : {"messages": [HumanMessage(…)]}
- réponse : resultat["messages"][-1].content

Même forme partout

create_agent, graphe à la main, graphe protégé (Ex1.2) : on les appelle tous ainsi.

⚠️ recursion_limit : la valeur par défaut a changé

Défaut LangGraph 1.2

10 007

étapes avant GraphRecursionError. Les tutoriels annoncent encore 25.

Un agent qui boucle peut enchaîner des milliers d'appels LLM.

```python
from langgraph.errors import GraphRecursionError

try:
    support.invoke(entree, config={"recursion_limit": 10})
except GraphRecursionError:
    # journaliser, prévenir un humain, répondre poliment
    ...
```

💡 Le « Tips » du Formateur

Fixez toujours recursion_limit explicitement (10 à 25 pour un agent de support). Constaté dans LangGraph 1.2 : DEFAULT_RECURSION_LIMIT = 10007, surchargeable via LANGGRAPH_DEFAULT_RECURSION_LIMIT. La doc publique affiche encore 1000 : le code fait foi.

Erreurs classiques avec StateGraph

Symptôme	Cause	Correctif
L'historique disparaît à chaque tour	messages sans reducer	MessagesState ou Annotated[list, add_messages]
Une clé renvoyée n'arrive pas dans l'état	clé absente du State : ignorée sans erreur	déclarer la clé dans le TypedDict
Le graphe ne s'arrête jamais	pas de chemin vers END	tools_condition / routeur vers END + recursion_limit
« Node not found » à la compilation	faute de frappe dans add_edge	utiliser des constantes pour les noms de nœuds
État modifié de façon imprévisible	mutation sur place (state[...] = ...)	renvoyer un nouveau dict partiel
Routage invisible dans le dessin	fonction de routage non typée	retour annoté Literal[...] ou path_map

create_agent ou StateGraph à la main ?

create_agent

- boucle agent / outils prête à l'emploi
- middlewares (HITL, résumé, limites, PII)
- idéal : un agent autonome standard
- limite : structure imposée

StateGraph explicite

- vous dessinez chaque étape et chaque décision
- garde-fous, routeurs, sous-graphes, parallélisme
- idéal : workflows métier, multi-agents
- limite : plus de code à maintenir

Les deux se combinent : un create_agent peut être un nœud d'un StateGraph plus large (module Multi-agents).

🧪 À vous : J1-03

14h00–15h15 · notebook J1_03_stategraph

- **§2** le premier graphe de tri, puis son dessin Mermaid
- **§3** invoke, invoke(version="v2") et stream (updates, values)
- **§4** reducers : sans / avec operator.add, le cas parallèle, add_messages
- **§5** le tri par LLM, tracé (session j1-03-tri)
- **§6** l'agent en graphe explicite avec ToolNode et tools_condition
- **§7** provoquer GraphRecursionError avec recursion_limit=8

Slides à revoir

- premier graphe 1/2 et 2/2
- reducers en code
- agent en graphe 1/2 et 2/2

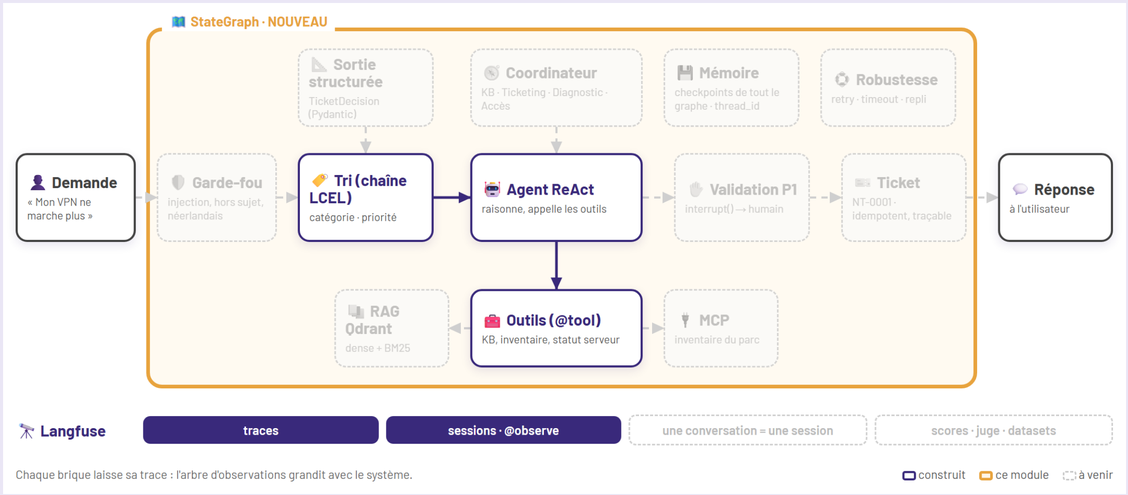
Ensuite

Exercice 1.1 : vous refaites l'agent en graphe, seuls, avec un 2e outil.

05

Construction dirigée

L'agent de support devient un graphe : l'étape 3, à la place de l'exercice 1.1.



Fiches : annexe p. 32–39

🛠️ Construction · étape 3 : le système devient un graphe

correction distribuée en fin d'étape : etape_03_stategraph.py · environ 30 min

```
lancer

$PY construction/atelier/mon_helpdesk.py \
    "srv-erp-02 est-il en ligne ?"
```

```
perdu ? repartir de la correction

# en fin d'étape, avec la correction distribuée :
cp etape_03_stategraph.py \
   construction/atelier/mon_helpdesk.py
$PY construction/atelier/mon_helpdesk.py
```

🧱 Ce qu'on ajoute

- l'état EtatTicket (MessagesState + tri)
- les nœuds trier ⟶ agent ⇄ tools
- le dessin du graphe (ASCII et Mermaid)

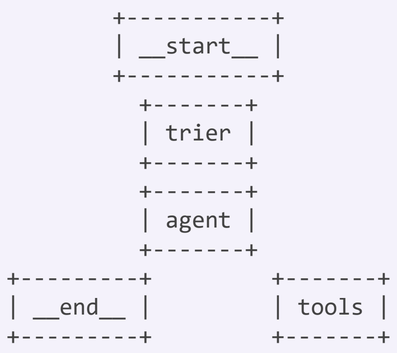
👀 À observer

- le graphe dessiné dans le terminal
- chaque nœud au moment où il s'exécute
- Langfuse : les enfants portent le nom des nœuds

Étape 3 : ce que le terminal affiche

Le graphe se dessine, puis chaque nœud s'annonce quand il s'exécute

▶ le graphe (draw_ascii, abrégé)



▶ l'exécution (stream_mode="updates", abrégé)

```
🎫 srv-vpn-01 est-il en ligne ?
   ▶ trier  : reseau · P3 · file N2-infra
   ▶ agent  : demande statut_serveur({'nom': 'srv-vpn-01'})
   ▶ tools  : en ligne
   ▶ agent  : répond « srv-vpn-01 est en ligne : … »
🔭 Langfuse › Tracing › helpdesk
```

Le schéma dessiné, l'ordre d'exécution et l'arbre de la trace racontent **la même histoire** : c'est ce qu'on vérifie à chaque étape.

Lire une trace : les 6 questions à se poser

1

Quel est le chemin suivi ?

→ arbre des observations

2

Quel prompt est réellement parti ?

→ input de la generation

3

Quel outil, avec quels arguments ?

→ observations tool

4

Où passe le temps ?

→ Gantt, latence par étape

5

Combien de tours de boucle ?

→ nombre de generations

6

Des données personnelles visibles ?

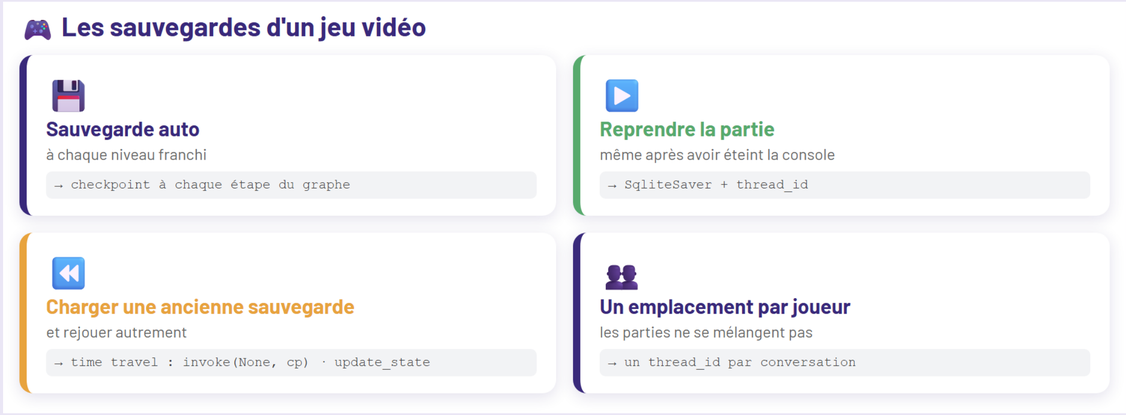
→ inputs et outputs masqués ?

06

Persistance

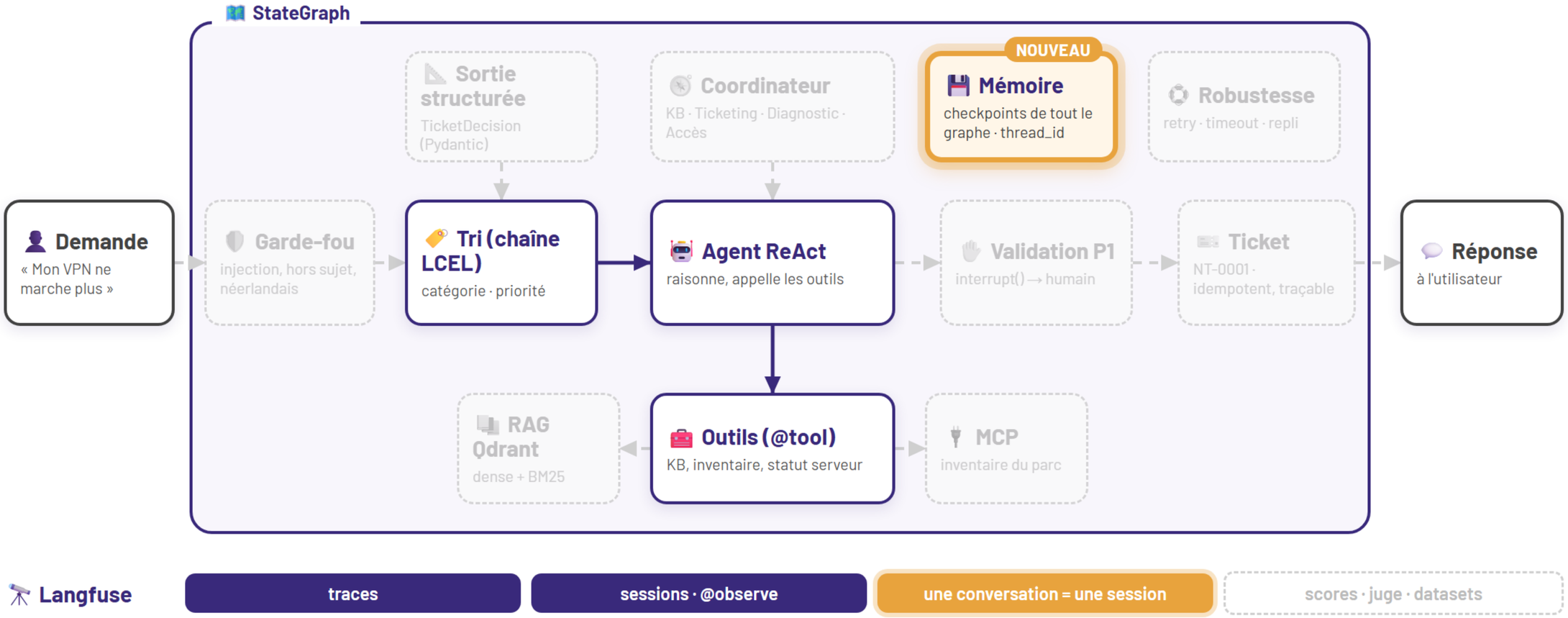
Checkpoints, threads, voyage dans le temps… et données personnelles en clair.

📎 Fiches : annexe p. 40–49



Où en est HelpDesk-Ops ?

On ajoute la mémoire : un checkpoint à chaque étape, un thread par conversation.



Chaque brique laisse sa trace : l'arbre d'observations grandit avec le système.

construit · ce module · à venir

Les sauvegardes d'un jeu vidéo

🎮 Les sauvegardes d'un jeu vidéo

💾

Sauvegarde auto

à chaque niveau franchi

```
→ checkpoint à chaque étape du graphe
```

▶️

Reprendre la partie

même après avoir éteint la console

```
→ SqliteSaver + thread_id
```

⏪

Charger une ancienne sauvegarde

et rejouer autrement

```
→ time travel : invoke(None, cp) · update_state
```

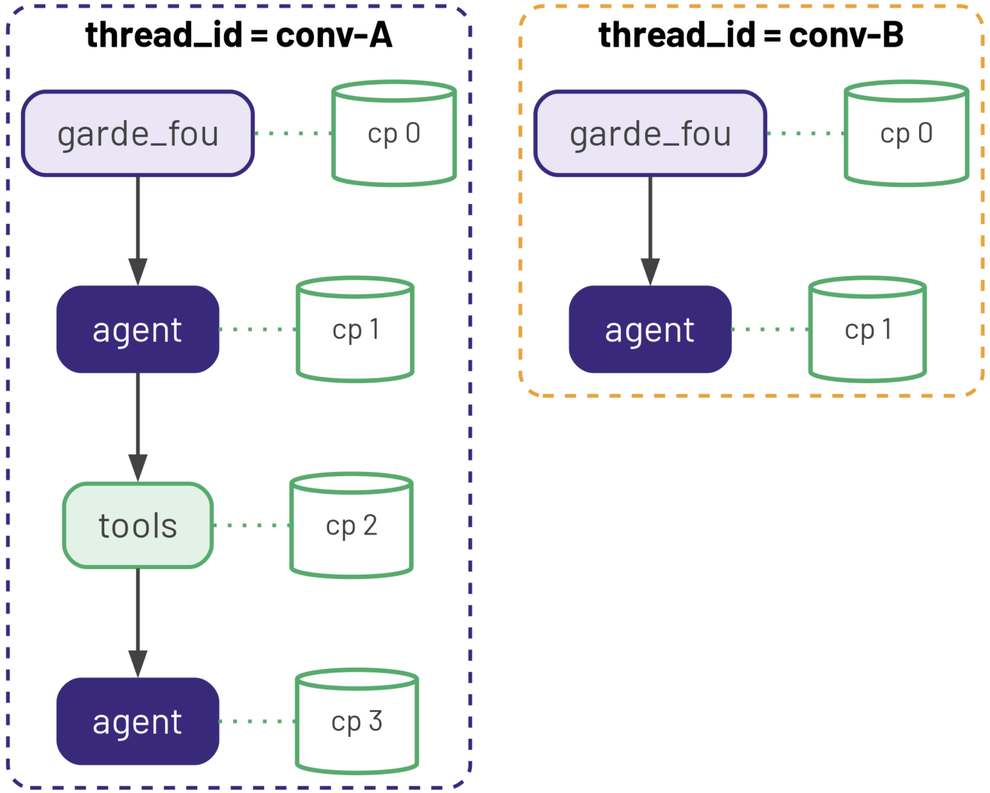
👥

Un emplacement par joueur

les parties ne se mélangent pas

```
→ un thread_id par conversation
```

Un checkpoint par étape, un thread par conversation



- **Checkpoint :** l'état complet + la suite prévue, écrit à la fin de chaque super-step
- **thread_id :** l'adresse d'une conversation, pas d'un utilisateur
- **Sans checkpointer :** chaque invoke repart de zéro, l'agent est amnésique
- **Ce que ça permet :** mémoire, reprise après panne, validation humaine, voyage dans le temps

Trois checkpointers, un même contrat

Checkpointer	Stockage	Survit au redémarrage	Usage
`InMemorySaver`	mémoire du processus	❌	tests, démonstrations
`SqliteSaver / AsyncSqliteSaver`	un fichier SQLite (data/checkpoints/)	✅	poste unique, formation
`PostgresSaver`	PostgreSQL	✅	production, plusieurs instances

```python
from langgraph.checkpoint.memory import InMemorySaver

support = construire_support(llm, checkpointer=InMemorySaver())
config = {"configurable": {"thread_id": "conv-A"}}
support.invoke({"messages": [("user", "srv-erp-02 est-il en ligne ?")]}, config)
support.invoke({"messages": [("user", "Et ce serveur, son nom exact ?")]}, config)
```

Lire l'état : get_state et get_state_history

```python
config_A = {"configurable": {"thread_id": "j2-01-conv-A"}}
etat = avec_memoire.get_state(config_A)
print("next     :", etat.next)        # () = terminé ; ('agent',) = en attente
print("messages :", len(etat.values["messages"]))

for snap in avec_memoire.get_state_history(config_A):   # du plus récent
    print(f"step {snap.metadata['step']:>2} | next={snap.next}")
```

▶ Sortie exacte : 2 tours, modèle factice sans outil (2 colonnes pour la slide)

```
next     : ()
messages : 4
step  6 | next=()                  step  2 | next=()
step  5 | next=('agent',)          step  1 | next=('agent',)
step  4 | next=('garde_fou',)      step  0 | next=('garde_fou',)
step  3 | next=('__start__',)      step -1 | next=('__start__',)
```

StateSnapshot

- values · next · config
- metadata (step, source)
- tasks (interruptions)

💡 Le « Tips » du Formateur

Un tour = plusieurs checkpoints (garde_fou, agent, et tools à chaque appel d'outil). Le step -1 est l'entrée brute.

SqliteSaver : la conversation survit au redémarrage

```python
import sqlite3
from langgraph.checkpoint.sqlite import SqliteSaver
from helpdesk_ops.persistance import chemin_checkpoints

connexion = sqlite3.connect(chemin_checkpoints("j2.sqlite"),
                            check_same_thread=False)
support = construire_support(llm, checkpointer=SqliteSaver(connexion))
# … redémarrage du kernel, nouvelle connexion au même fichier …
# la conversation reprend là où elle s'était arrêtée
```

Deux tables

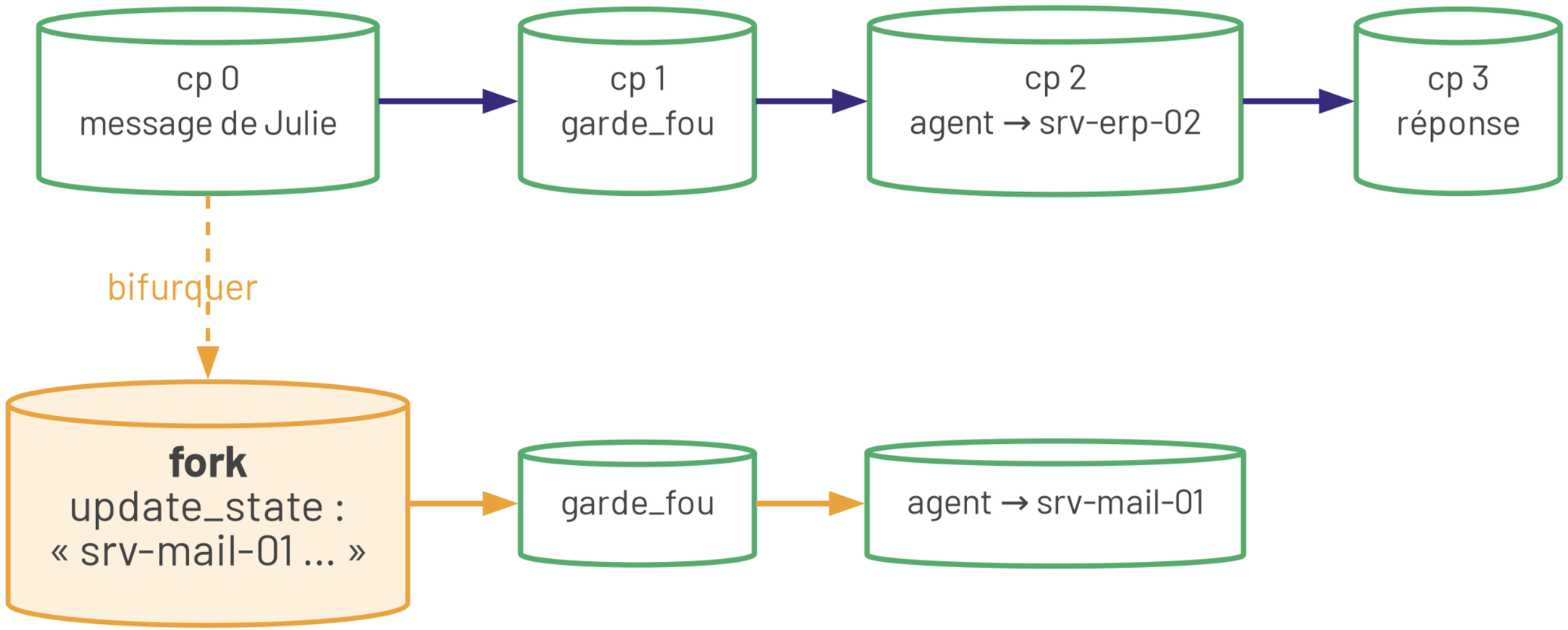
- checkpoints : les états
- writes : écritures intermédiaires

💡 Le « Tips » du Formateur

Le fichier est rangé dans data/checkpoints/ et ignoré par Git : il contient des conversations. En asynchrone : AsyncSqliteSaver + aiosqlite.

Voyage dans le temps : rejouer et bifurquer

Rejouer : invoke(None, cp) · Bifurquer : update_state(cp, valeurs)



Bifurquer : corriger un message et relancer

```
historique = list(support.get_state_history(config))
avant = [h for h in historique if h.next == ("garde_fou",)][-1]
origine = avant.values["messages"][-1]

# même id → add_messages REMPLACE le message (vu en J1-03)
corrige = HumanMessage("srv-mail-01 ne répond plus", id=origine.id)
fork = support.update_state(avant.config, {"messages": [corrige]})
support.invoke(None, fork)          # relance depuis la branche
```

⚠️ Rejouer n'est pas gratuit

Les nœuds relancés rappellent le LLM… et les outils.

Un outil qui écrit, rejoué deux fois = un doublon.

→ **idempotence**

🔒 RGPD : un checkpoint contient la conversation en clair

Le masquage Langfuse du jour 1 ne s'applique qu'aux traces

```
connexion.execute(
    "SELECT COUNT(*) FROM checkpoints "
    "WHERE thread_id = ? AND instr(checkpoint, ?) > 0",
    ("j2-01-julie", b"julie.martin@nordtech.example"))
# → 10 checkpoints contiennent l'e-mail EN CLAIR
```

10

checkpoints sur 17 portaient l'e-mail de Julie (préparation, 20/10/2026)

Obligation	Pour les checkpoints
Conservation limitée (art. 5.1.e)	purge planifiée des conversations inactives : purger_threads(saver, timedelta(days=...))
Droit à l'effacement (art. 17)	saver.delete_thread(thread_id) : checkpoints ET écritures
Minimisation (art. 5.1.c)	⚠️ prune(..., "keep_latest") est documenté mais NON implémenté par SqliteSaver 3.1

Une purge qui s'applique vraiment

```python
from datetime import timedelta
from helpdesk_ops.persistance import purger_threads

# 1. simulation : à journaliser avant la première vraie purge
purger_threads(saver, timedelta(days=30), simulation=True)

# 2. en tâche planifiée (cron), durée écrite au registre des traitements
purger_threads(saver, timedelta(days=30))

# 3. demande d'effacement d'une personne
saver.delete_thread("j2-01-julie")
```

Effacée partout ?

- checkpoints : ✅
- traces Langfuse : masquées, mais le nom passait (J1)
- tickets : le résumé
- sauvegardes, journaux applicatifs

🧪 À vous : J2-01

08h30–09h45 · notebook J2_01_persistance

- **§2** l'agent amnésique : deux questions sans checkpointer
- **§3** InMemorySaver : deux conversations A et B, isolées par thread_id
- **§4** get_state et get_state_history : compter les checkpoints
- **§5** SqliteSaver : la conversation survit au « redémarrage »
- **§6 – 7** bifurquer depuis un checkpoint, puis chercher l'e-mail en clair et effacer
- **Mini-défi :** exporter_conversation(graphe, thread_id), le droit d'accès

Slides à revoir

- trois checkpointers
- lire l'état · bifurquer
- RGPD · purge

Le fil rouge

construire_support(llm, checkpointer=…) : l'agent du jalon J1, rendu stateful.

🛠️ Construction · étape 4 : la mémoire des conversations

correction distribuée en fin d'étape : etape_04_memoire.py · environ 10 min

```
lancer

$PY construction/atelier/mon_helpdesk.py --thread conv-42 "Mon VPN ne marche plus"
$PY construction/atelier/mon_helpdesk.py --thread conv-42 "Et srv-vpn-01 ?"
$PY construction/atelier/mon_helpdesk.py --thread conv-42 --oublier
```

```
perdu ? repartir de la correction

# en fin d'étape, avec la correction distribuée :
cp etape_04_memoire.py \
   construction/atelier/mon_helpdesk.py
$PY construction/atelier/mon_helpdesk.py
```

🧱 Ce qu'on ajoute

- SqliteSaver : un checkpoint par étape
- l'option thread : la conversation
- l'option oublier : le droit à l'effacement

👀 À observer

- 2e lancement : l'agent se souvient du 1er
- messages et checkpoints qui augmentent
- Sessions : session = thread

07

Human-in-the-loop

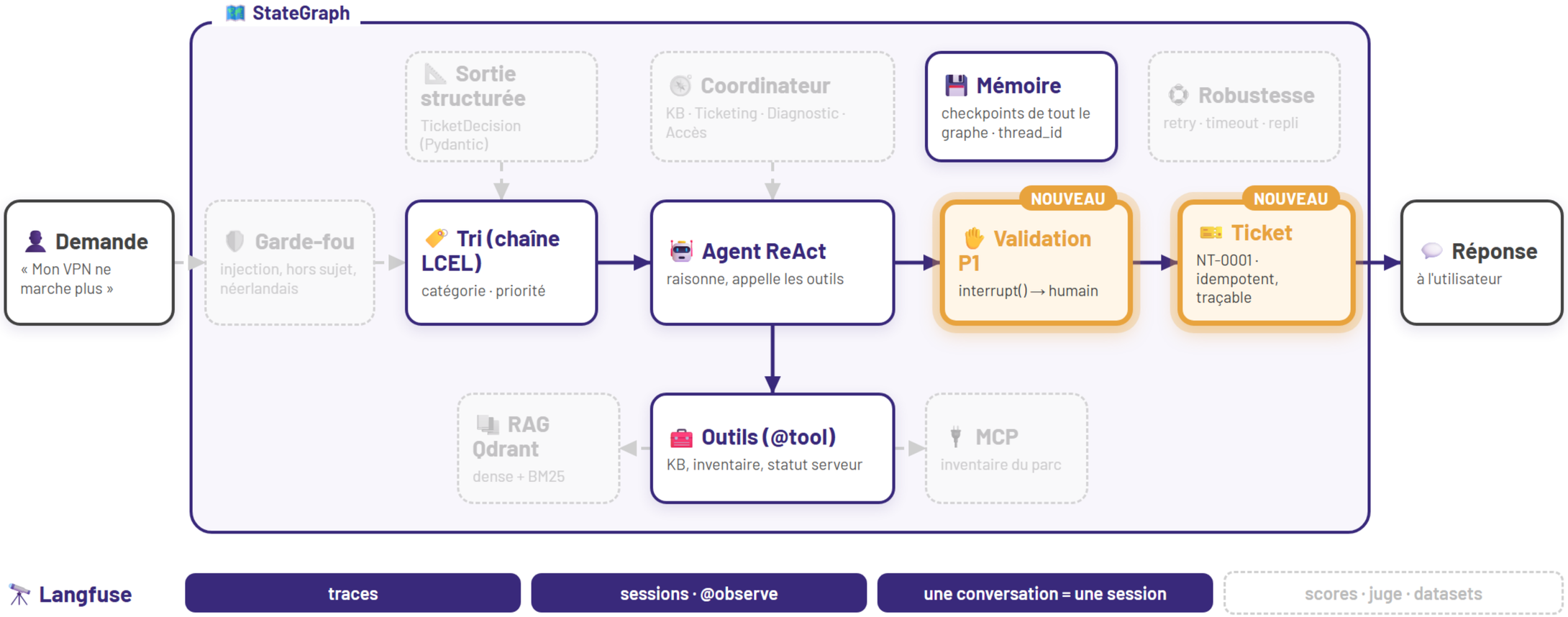
L'agent propose, l'humain dispose : interrupt() et Command(resume=...).



Fiches : annexe p. 50–55

Où en est HelpDesk-Ops ?

On ajoute la validation humaine des P1 et la création du ticket.



Chaque brique laisse sa trace : l'arbre d'observations grandit avec le système.

construit · ce module · à venir

La signature du responsable

✍️ La signature du responsable



Le circuit **attend** sans rien perdre, puis reprend exactement où il en était.

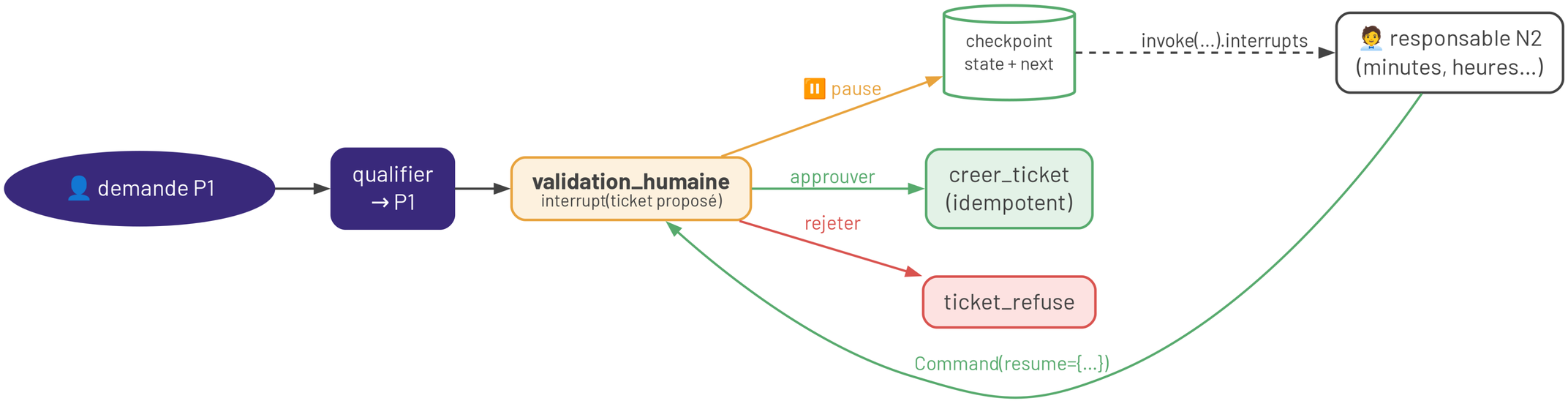
Pourquoi une validation humaine ?

	Sans HITL	Avec HITL
Décision	l'agent décide et agit seul	l'agent propose, un humain dispose
Erreur de priorité	l'astreinte est réveillée à 3 h pour rien	le responsable corrige P1 → P2 avant création
Responsabilité	floue	tracée : qui a validé, quand, quoi
Réglementaire	—	supervision humaine (AI Act art. 14 pour le haut risque)

💡 **Le « Tips » du Formateur**

Toutes les actions ne méritent pas une validation : on la réserve à ce qui est irréversible ou coûteux (P1, redémarrage, envoi externe). Trop de validations = fatigue = validations aveugles.

La mécanique : pause, sauvegarde, reprise



interrupt() (1/2) : suspendre le graphe

Sans LLM : la mécanique est la même, et l'exécution instantanée

```python
from langgraph.types import Command, interrupt

class Demande(TypedDict):
    resume: str
    priorite: str
    decision: str

def valider(state: Demande) -> dict:
    reponse = interrupt({"question": f"Créer le ticket {state['priorite']} "
                                     f"« {state['resume']} » ?"})
    return {"decision": reponse}        # reponse = la valeur de la reprise

# graphe : START -> valider -> executer -> END
mini = b.compile(checkpointer=InMemorySaver())   # checkpointer obligatoire
config = {"configurable": {"thread_id": "j2-02-mini"}}
sortie = mini.invoke({"resume": "ERP de production arrêté", "priorite": "P1"},
                     config, version="v2")
print("Interruption :", sortie.interrupts[0].value)
print("En attente de :", mini.get_state(config).next)
```

▶ **Sortie exacte**

```
Interruption : {'question': 'Créer le ticket P1 « ERP de production arrêté » ?'}
En attente de : ('valider',)
```

Trois règles

- interrupt(valeur) suspend et rend valeur à l'appelant
- il faut un checkpointer et un thread_id
- version="v2" : .value et .interrupts

interrupt()(2/2) : reprendre avec Command(resume=...)

```python
# même thread : le graphe repart du nœud valider, interrupt() renvoie la réponse
sortie = mini.invoke(Command(resume="approuvé par resp-n2"), config, version="v2")
print("État final :", sortie.value)
print("Interruptions restantes :", sortie.interrupts)

# reprendre une 2e fois un thread terminé ?
sortie = mini.invoke(Command(resume="approuvé par resp-n2"), config, version="v2")
print("next :", mini.get_state(config).next)
```

▶ Sortie exacte (la 2e reprise n'affiche rien : executer ne repasse pas)

```
   ⚙️  exécution : décision = approuvé par resp-n2
État final : {'resume': 'ERP de production arrêté', 'priorite': 'P1',
              'decision': 'approuvé par resp-n2'}
Interruptions restantes : ()
next : ()
```

À retenir

- rien n'est perdu pendant l'attente
- reprise possible depuis un autre processus (SQLite)

💡 Le « Tips » du Formateur

Thread terminé : une reprise ne relance rien. Le vrai danger du doublon, c'est le code AVANT interrupt()(slide suivante).

⚠️ Le piège du rejeu : le nœud repart du début

```python
def valider_et_creer_MAUVAIS(state):
    compteur["tickets"] += 1          # ❌ effet de bord AVANT interrupt
    reponse = interrupt({"question": "Valider ?"})
    return {"decision": reponse}

graphe.invoke(entree, config)
graphe.invoke(Command(resume="ok"), config)
# compteur["tickets"] == 2 pour UNE demande
```

2

tickets pour une seule demande

Parade
rien avant interrupt()
action dans un nœud séparé
clé d'idempotence (thread_id)

Le ticketing simulé : une clé d'idempotence

helpdesk_ops.ticketing : créer deux fois le ticket d'une même demande renvoie le ticket existant

```python
from helpdesk_ops.ticketing import Ticketing

ticketing = Ticketing()                          # vide au départ
for _ in range(3):                               # trois rejeux de la même demande
    ticket = ticketing.creer(cle_idempotence="j2-02-rejeu",
                             categorie="logiciel", priorite="P1",
                             resume="ERP arrêté", file="N1",
                             valide_par="resp-n2")   # facultatif

print("Tickets en base :", [t.id for t in ticketing.lister()])
print(ticket.id, ticket.priorite, ticket.valide_par)
print(ticketing.par_cle("j2-02-rejeu").id, ticketing.par_cle("inconnue"))
```

▶ **Sortie exacte**

```
Tickets en base : ['NT-0001']
NT-0001 P1 resp-n2
NT-0001 None
```

L'API

- creer(*, cle_idempotence, categorie, priorite, resume, file, valide_par=None)
- lister() · par_cle(clé) · vider()

💡 Le « Tips » du Formateur

Les arguments sont nommés (après *) : ticketing.creer("x", …) lève une TypeError.

Un nœud qui connaît sa conversation : config: RunnableConfig

Exemple : journaliser le thread_id. Le même paramètre donne la clé d'idempotence d'une action

```python
from langchain_core.runnables import RunnableConfig

def journaliser(state: Etat, config: RunnableConfig) -> dict:
    thread_id = config["configurable"]["thread_id"]   # le config de l'appel
    return {"trace": f"{thread_id} : {state['texte']}"}

# graphe : START -> journaliser -> END, compilé avec un checkpointer
g.invoke({"texte": "demande reçue"},
         {"configurable": {"thread_id": "ex-voisin-a"}})
```

▶ **Sortie exacte**

```
{'texte': 'demande reçue', 'trace': 'ex-voisin-a : demande reçue'}
```

Comment ça marche

LangGraph repère le paramètre nommé config (annoté RunnableConfig) et y passe le config de l'invoke.

Pourquoi le thread_id ?

Une demande = un thread : c'est une clé d'idempotence naturelle.

Des décisions riches, et validées

```python
def validation_humaine(state) -> Command[Literal["creer", "refuser"]]:
    question = {"ticket_propose": {...}, "actions": ACTIONS}
    while True:          # chaque interrupt() attend sa propre reprise
        reponse = interrupt(question)
        if reponse.get("action") in ACTIONS and reponse.get("par"):
            break
        question = {**question, "erreur": f"réponse invalide : {reponse!r}"}
    if reponse["action"] == "rejeter":
        return Command(update={"validation": reponse}, goto="refuser")
    maj = {"validation": reponse}
    if reponse["action"] == "modifier":
        maj["priorite"] = reponse["priorite"]
    return Command(update=maj, goto="creer")
```

💡 Le « Tips » du Formateur

La réponse humaine vient d'une interface, d'un mail, d'un chat : c'est une ENTRÉE. On la valide comme les autres.

HITL dans create_agent : le HumanInTheLoopMiddleware

```python
from langchain.agents.middleware import HumanInTheLoopMiddleware

agent = create_agent(
    model=llm, tools=[statut_serveur, creer_ticket],
    checkpointer=InMemorySaver(),
    middleware=[HumanInTheLoopMiddleware(
        interrupt_on={"creer_ticket": True, "statut_serveur": False})],
)
s = agent.invoke(entree, config, version="v2")
s.interrupts[0].value["action_requests"]   # nom + arguments exacts

agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}), config)
```

Décision
approve
edit
reject
respond

Dans Langfuse : une validation = deux traces, une session



Maquette pédagogique de l'interface Langfuse v4 (disposition simplifiée)

🛠️ Construction · étape 5 : la validation humaine et le ticket

correction distribuée en fin d'étape : etape_05_validation_ticket.py · environ 30 min

```
lancer

$PY construction/atelier/mon_helpdesk.py --thread t-vpn \
    "Le VPN est bloqué pour tout le service, urgent"
$PY construction/atelier/mon_helpdesk.py --thread t-vpn --approuver marie
```

```
perdu ? repartir de la correction

# en fin d'étape, avec la correction distribuée :
cp etape_05_validation_ticket.py \
   construction/atelier/mon_helpdesk.py
$PY construction/atelier/mon_helpdesk.py
```

🧱 Ce qu'on ajoute

- valider : interrupt() pour les P1
- creer_ticket : idempotent, dans un fichier JSON
- les options approuver ou refuser : la reprise

👀 À observer

- ⏸ le graphe s'arrête, le terminal se ferme
- la reprise, plus tard, crée NT-0001
- Sessions : deux traces, une conversation

Étape 5 : la pause, puis la reprise

Deux lancements, deux terminaux si l'on veut : le checkpoint fait le lien

▶ 1er lancement : le graphe s'arrête (abrégé)

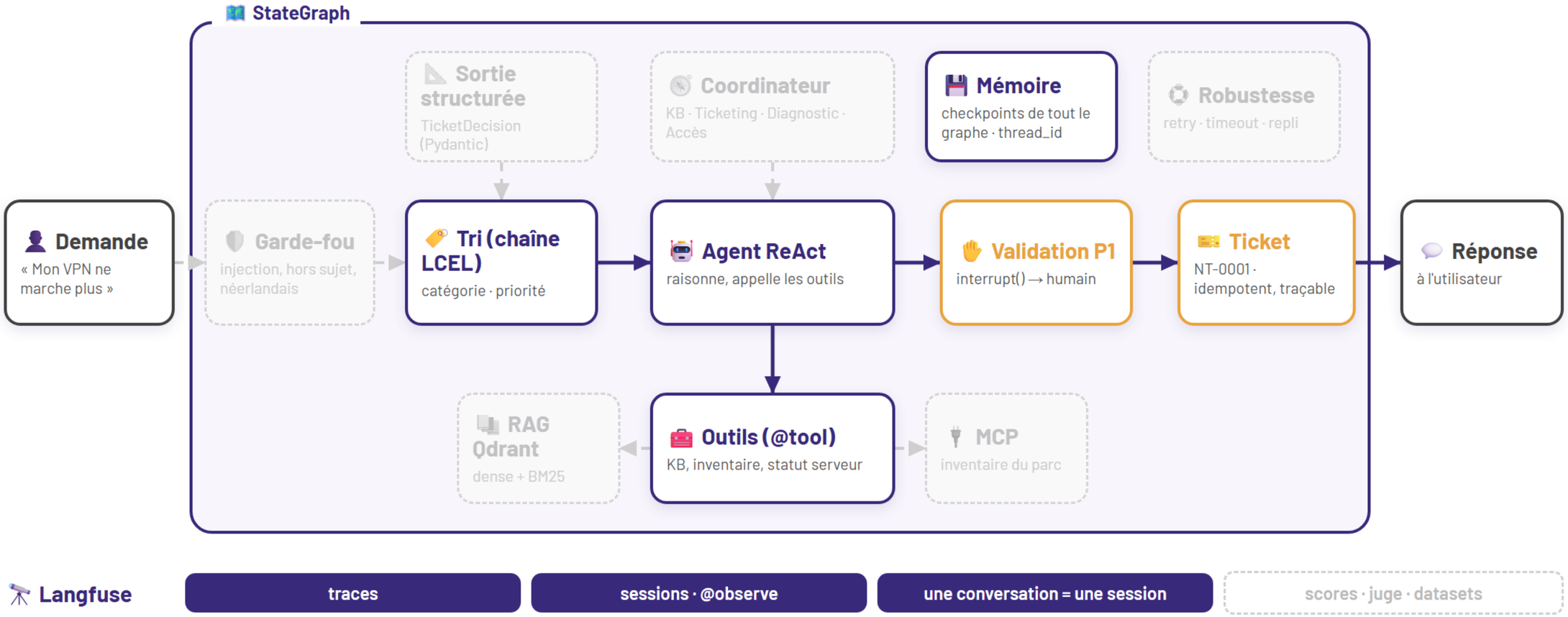
```
$ ... mon_helpdesk.py --thread t-vpn "Le VPN est bloqué pour tout le service, urgent"
🎫 Le VPN est bloqué pour tout le service, urgent
   🧰 outil appelé : statut_serveur({'nom': 'srv-vpn-01'})
   🏷️  reseau · P1 · file N2-infra
⏸  Créer ce ticket P1 ? Le graphe est arrêté, son état est dans le checkpoint.
```

▶ plus tard : la décision humaine reprend le graphe

```
$ ... mon_helpdesk.py --thread t-vpn --approuver marie
✍️  Décision de marie : approuvé
   🎫 Ticket NT-0001 créé dans la file N2-infra, validé par marie.
📒 Tickets : data/checkpoints/construction_tickets.json
```

Ce soir, HelpDesk-Ops…

… trie, raisonne, se souvient, consulte un humain et crée le ticket. Lundi : l'étape 6.



Chaque brique laisse sa trace : l'arbre d'observations grandit avec le système.

construit · ce module · à venir

Lundi 09/11 : la suite

🛠️ Construction · étape 6

la sortie structurée et la robustesse : TicketDecision, RetryPolicy

📚 RAG en 3 briques

embeddings, Qdrant, retriever : la KB devient interrogeable

🧭 Coordinateur et spécialistes

KB-RAG, Ticketing, Diagnostic, Accès, exposés comme outils

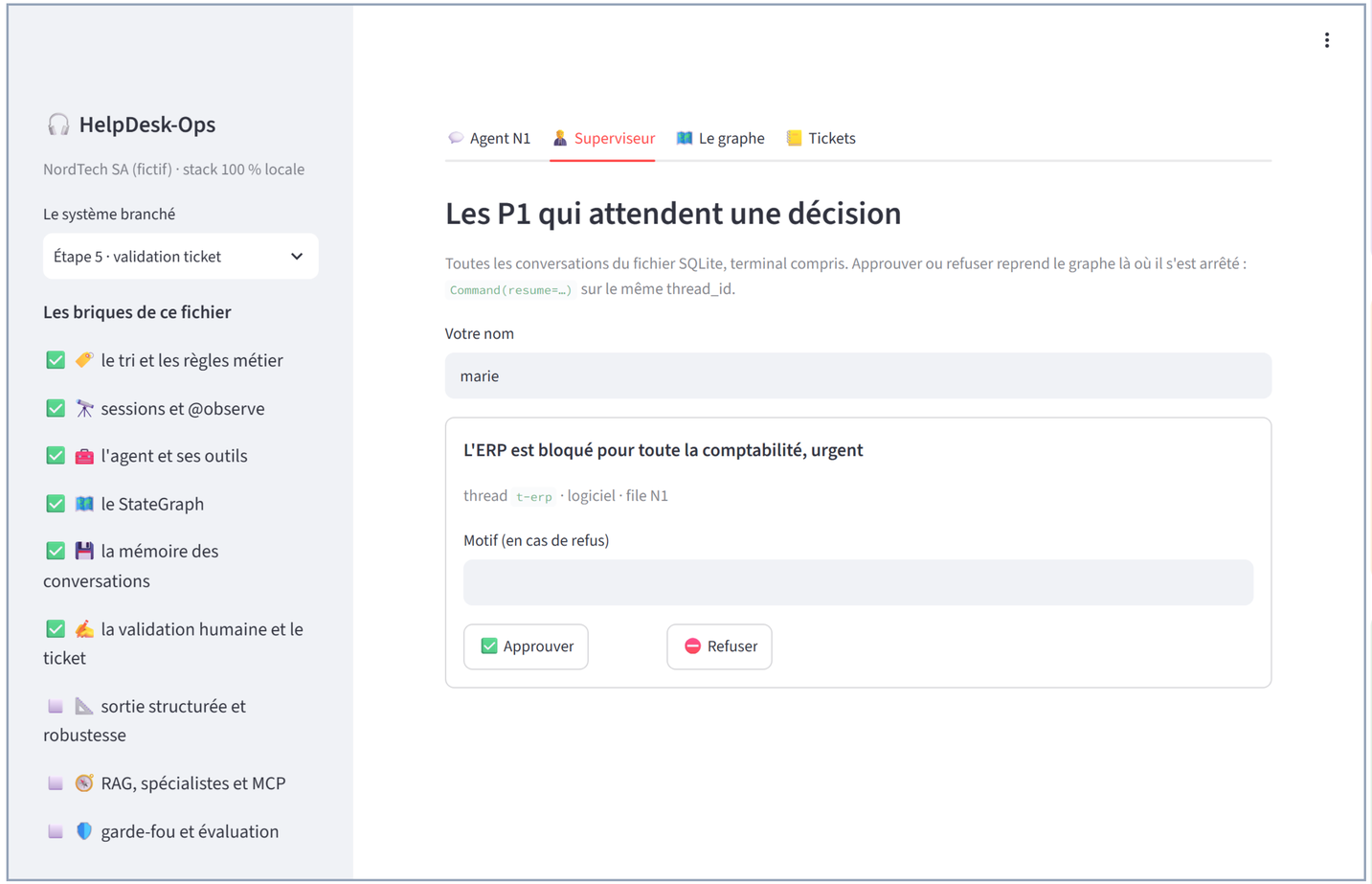
🔭 Observabilité

l'arbre de traces multi-agents, lu par l'API metrics

Ce soir : les postes peuvent rester éteints, la stack redémarre seule. Votre atelier/mon_helpdesk.py reste sur le poste : lundi, on repart de l'étape 5.

Lundi : votre graphe devient un outil

La console HelpDesk-Ops branche VOTRE mon_helpdesk.py : une interface, aucune ligne de plus



🧱 Ce que vous écrivez

- l'état, les nœuds, le builder : le graphe
- la console ne touche pas à votre fichier

🗺️ Ce qu'elle affiche

- les briques présentes, et seulement elles
- le nœud qui s'exécute, allumé

👷 Le superviseur

- les P1 en pause, terminal compris
- Approuver : Command(resume=...)

“ Citation “

« Le code devient un schéma, et le schéma devient une trace. »

Fil rouge de la formation : dessiner, exécuter, observer

Merci · Questions ?

Lundi : RAG, multi-agents et observabilité

Ressources : docs.langchain.com · langfuse.com/docs · dossier Orchestration/ du cours

